FACULTY OF INFORMATION TECHNOLOGY

IT PROJECT 700 | GROUP 55

EMERGEX

Centralized Student Accommodation & Secure Booking Application

FINAL INTEGRATED PROJECT REPORT

Complete SDLC documentation and implementation evidence

Supervisor: Thabiso | ThabisoA@richfield.ac.za | Semester 2, 2026

Student no.	Member	Email	Contact	Primary role
402308936	Tristan Moses	402308936@my.richfield.ac.za	0828015082	Project manager and frontend lead
402411668	Reenesh Ramawthar	402411668@my.richfield.ac.za	0846041521	Backend and quality engineering
402411227	Joyce Kamalu	402411227@my.richfield.ac.za	0671075365	Business analysis and documentation
402412304	Novithan Naidoo	402412304@my.richfield.ac.za	0620829186	Database and backend engineering
402413409	Thingo Shezi	402413409@my.richfield.ac.za	0714299251	UI/UX and frontend engineering
402111846	Lungelo Ngwane	402111846@my.richfield.ac.za	0642534563	Testing, deployment and support

Project lead baseline: Tristan Moses | Every member contributes and presents.

Document Control

Table 1. Document control

Field	Value
Document owner	Group 55
Version	1.0 - assessment baseline
Prepared	18 August 2026
Approval status	For supervisor review
Confidentiality	Academic project; demo data only
Repository	Group 55 - Completed Project/application

Academic Integrity and Contribution Statement

The report represents a unified group deliverable. The responsibilities shown on the administrative page identify primary owners, not exclusive authors. All six members are expected to review the final report, understand the implementation, participate in supervisor meetings and present their assigned section to the assessment panel.

Executive Summary

EMERGEX is a secure, centralized web application prototype for South African students seeking accommodation near tertiary institutions. The project responds to a long-standing supply and quality problem documented by the Department of Higher Education and Training, and to the practical difficulty of comparing trustworthy listings, obtaining timely landlord responses and managing deposit risk. The solution brings property discovery, verification evidence, booking status and a simulated deposit-hold lifecycle into one auditable workflow.

The implementation uses a LAMP architecture: Apache and PHP provide server-side application logic, MySQL/InnoDB enforces relational integrity, while HTML5, CSS3 and JavaScript deliver a responsive interface. Three user roles are implemented. Students search, filter, book, upload NSFAS evidence and confirm check-in. Landlords manage verified listings and respond to requests. Administrators review identities, properties and documents and monitor audit events. The prototype deliberately does not process real money; production escrow would require a regulated payment provider and formal legal review.

A hybrid SDLC is applied. Waterfall gates align the academic phases and approval dates, while Agile sprints support parallel delivery across frontend, backend, database, analysis, documentation and testing work. The final package contains the runnable application, schema, seed data, installation guide, security checks, phase reports, this integrated report and an assessment presentation.

Table of Contents

Document Control 3
Academic Integrity and Contribution Statement 3
Executive Summary 3
Abbreviations and Key Terms 6
Project Purpose and Direction 7
Purpose of the Project 7
Objectives 7
Implementation Language and Platform 8
SDLC Model 8
Planning Phase 10
Identification of Need 10
Preliminary Investigation 11
Feasibility Study 11
Project Charter and Scope 11
Work Breakdown and Responsibilities 13
Project Schedule 14
Risk Management 16
Communication, Quality and Change Control 16
Software Requirements Specification Summary 17
Software Engineering Paradigm 18
Analysis Phase 19
Information-Gathering Methodology 19
Analysis of the Existing System 19
Weaknesses and Root Causes 20
Data Analysis: Integrity and Constraints 20
Analysis of the Proposed System 21
Functional Requirements 22
Non-Functional Requirements 23
Data Modelling 24
Requirements Validation and Traceability 25
System Design Phase 26
Description of the Proposed System 26
Architectural Design 27
Hardware and Network Architecture 28
Class and Behaviour Design 29
Physical and Database Design 30
Normalization 32
Program Design and Pseudocode 32

Interface Design 33
Input Validation Design 33
Output and Report Design 34
Security and Backup Design 34
Design Test Cases 35
Implementation Phase 36
Implementation Overview 36
Coding Standards and Efficiency 36
Representative Code Segments 37
Error Handling and Validation 37
Installation 38
Operational Demonstration Script 38
Testing and Evaluation 40
Testing Strategy 40
Environment and Evidence Status 41
Detailed Test Register 42
Static Security Check Results 42
Defect Management 43
Evaluation of Results 43
System Security Measures 44
Database and Data Security 44
User Profiles and Access Rights 44
POPIA-Oriented Privacy Design 44
Project Cost Estimation 46
Economic Benefit Model 46
Reports and Sample Layouts 48
Future Scope and Enhancement 49
Maintenance and Configuration Management 49
Bibliography and Source Register 50
Glossary 51
Meeting and Review Templates 52
Final Submission Checklist 53

Abbreviations and Key Terms

Table 2. Abbreviations used in the report

Abbreviation	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
CSRF	Cross-Site Request Forgery
DFD	Data-Flow Diagram
ERD	Entity-Relationship Diagram
LAMP	Linux, Apache, MySQL and PHP
NSFAS	National Student Financial Aid Scheme
OWASP	Open Worldwide Application Security Project
PDO	PHP Data Objects
PERT	Program Evaluation and Review Technique
POPIA	Protection of Personal Information Act
RBAC	Role-Based Access Control
SDLC	Systems Development Life Cycle
SRS	Software Requirements Specification
UAT	User Acceptance Testing
WBS	Work Breakdown Structure
XSS	Cross-Site Scripting

PROJECT PROPOSAL

Project Purpose and Direction

This chapter defines the problem, intended value, technology choice and SDLC model approved as the foundation for subsequent work.

Purpose of the Project

South African tertiary institutions face a persistent student-housing capacity challenge. The 2011 ministerial review estimated a residence-bed shortage of 195,815 against recommended capacity targets, while a later DHET address described a projected university shortage of 216,000 beds and a separate immediate need of approximately 100,000 TVET beds [1][2]. Capacity alone does not solve the student journey: available rooms must also be discoverable, comparable, legitimate and responsive to a time-sensitive academic calendar.

EMERGEX centralizes this journey. It allows a student to search verified listings by city, suburb, nearby institution, price, NSFAS status and Instant Book availability. Each approved listing presents consistent rent, deposit, availability, amenities and visual-evidence fields. The booking workflow records who requested a room, the required move-in date, landlord decision, deposit status and check-in confirmation. Administrators can verify landlords, properties and uploaded evidence and can inspect an audit history of critical actions.

The prototype is positioned as a safer and more transparent alternative to fragmented social-media searches and informal direct payments. It does not claim that every competing marketplace lacks verification; DigsConnect, for example, currently advertises verified listings, in-platform or WhatsApp-led contact, and a landlord facilitation model [4][5]. EMERGEX differentiates its academic design through a single visible booking state machine, mandatory visual-evidence categories, an explicit 12-hour response target, an Instant Book path and a deposit-hold simulator tied to check-in confirmation.

Ethical and legal boundary
The academic system records simulated escrow statuses only. No real money is collected, held or transferred. Production release would require a licensed payment service provider, legal advice, formal POPIA governance and penetration testing.

Objectives

Table 3. Project objectives and success measures

Objective	Measurable success indicator
Centralize trusted discovery	A student can search and filter all published verified properties from one interface.
Reduce uncertainty	Every published property contains consistent location, price, deposit, amenities, availability and visual-evidence indicators.
Accelerate placement	Instant Book approves eligible requests immediately; standard requests store a 12-hour response deadline.
Demonstrate deposit protection	The prototype records not-paid, held, released and refunded states without processing funds.
Protect access	Student, landlord and administrator operations are restricted on the server by role and ownership.
Preserve evidence	Critical registration, booking, verification and escrow actions create timestamped audit records.
Support assessment	The system, documentation, test design and presentation trace directly to the five rubric

Objective	Measurable success indicator
	phases.

Implementation Language and Platform

The application is implemented with PHP 8, MySQL/MariaDB, Apache, HTML5, CSS3 and unobtrusive JavaScript. PHP was selected because the course concept explicitly targets a LAMP/XAMPP environment and because it offers built-in session management, password hashing, file handling and PDO database access. MySQL/InnoDB provides foreign keys, check constraints, transactions and indexes required by the booking and escrow state model. The presentation tier remains standards-based and does not depend on a large client framework, reducing local setup complexity.

Table 4. Technology selection rationale

Layer	Selection	Rationale
Client	HTML5, CSS3, JavaScript	Responsive, accessible and easy to demonstrate on desktop or mobile without a build step.
Server	Apache + PHP 8	Compatible with XAMPP; supports sessions, routing, uploads, validation and secure password APIs.
Data	MySQL / MariaDB with InnoDB	Relational integrity, transactions, indexes and broad academic familiarity.
Local environment	XAMPP on Windows	Repeatable classroom installation and one control panel for Apache and MySQL.
Testing	PHP security checks plus manual/system cases	Provides static control evidence and repeatable role-based acceptance scenarios.

SDLC Model

A hybrid Waterfall-Agile model is used. Waterfall provides the macro structure because the supervisor must approve sequential academic phases: proposal, planning, analysis, system design and implementation. Entry and exit criteria prevent unapproved scope from silently flowing into code. Within each approved phase, Agile execution divides the work into short sprints and component ownership. This allows frontend, backend, database, documentation and testing tasks to progress concurrently while still converging at scheduled integration reviews.

Table 5. Hybrid SDLC governance

Waterfall gate	Agile activity inside the gate	Exit evidence
Proposal	Concept review and feasibility spike	Approved purpose, stack and SDLC
Planning	Backlog refinement, estimates and risk review	SRS, WBS, Gantt, PERT and baseline
Analysis	Interviews, competitor review and model iteration	Validated functional/non-functional requirements
Design	Architecture, schema, UI and threat-model workshops	Design package and traceability review
Implementation	Feature sprints, integration and defect triage	Runnable build, tests, installation package

The approach balances predictability with learning. Requirements that affect the academic scope are controlled at phase gates, while implementation details may be refined inside sprints when tests or

stakeholder feedback reveal a better solution. Any material scope, schedule or security change is recorded in the change log and reviewed by the project lead and supervisor.

PHASE 2

Planning Phase

Planning converts the approved concept into a feasible, scheduled and testable delivery baseline.

Identification of Need

The need is for a trustworthy student-accommodation workflow that is usable during high-pressure registration periods. The primary user must be able to find available rooms near an institution, understand the complete financial commitment, establish whether the property is verified or NSFAS-accredited, and receive a timely decision. Landlords need qualified enquiries and an organized response queue. Administrators need evidence-based verification and traceable interventions rather than informal approval through messages or spreadsheets.

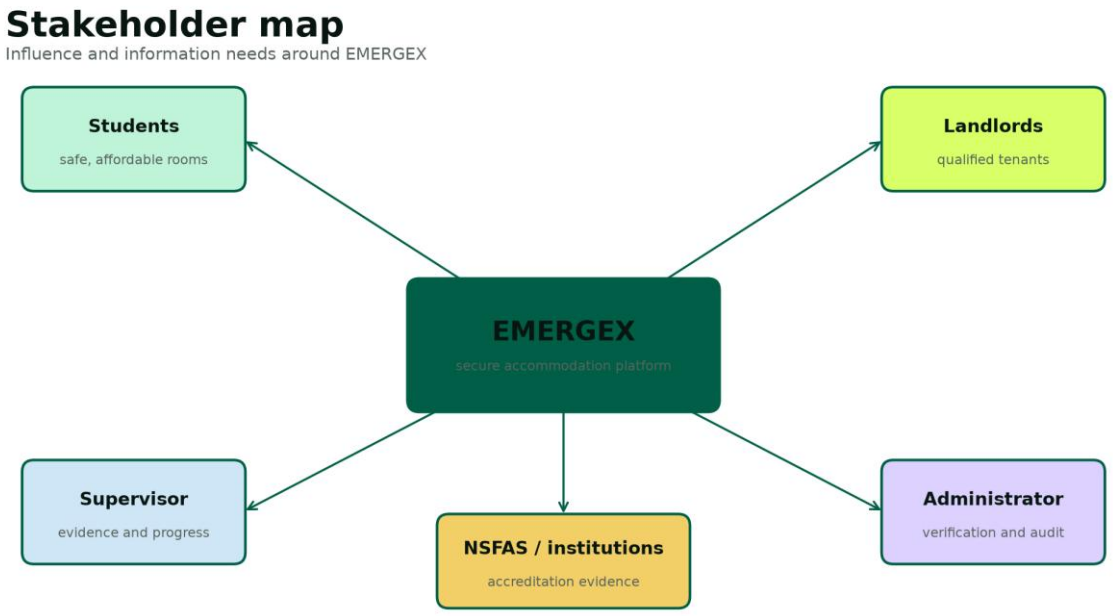


Figure 1. Stakeholder influence and information needs.

Table 6. Stakeholder register

Stakeholder	Interest	Influence	Information need	Engagement
Students	Safe, affordable and available accommodation	High	Price, distance, accreditation, visual proof, booking state	Usability reviews and acceptance tests
Landlords	Occupancy and credible applicants	High	Student request, move-in date, response deadline, deposit milestone	Workflow demos and dashboard review
Administrator	Platform trust and compliance	High	Verification queue, uploaded evidence and audit history	Weekly control review
Supervisor	Academic feasibility and evidence	High	Phase outputs, progress, risks and contribution	Scheduled reviews and approval gates
NSFAS / institutions	Accredited placement	Medium	Accreditation status and supporting evidence	Requirements validation; no live integration in prototype
Group members	Fair contribution and successful delivery	High	Backlog, owners, blockers and quality criteria	Twice-weekly stand-up and recorded minutes

Preliminary Investigation

The preliminary investigation combined the semester project guidelines, the existing Group 55 proposal, the EMERGEX pitch notes, official public information and an inspection of current accommodation marketplace behaviour. The investigation identified five recurring gaps to address in the prototype: fragmented discovery, uneven listing detail, uncertain verification, slow or invisible response status, and unsafe deposit behaviour outside a controlled workflow.

Table 7. Preliminary findings

Finding	Evidence or observation	Design response
Housing demand exceeds institutional supply	DHET reports document significant university and TVET accommodation shortfalls [1][2].	Support private and institutional listings in a common search model.
Accreditation matters for funded students	NSFAS states that funded students should use accredited accommodation and that unaccredited providers may not be paid [3].	Store and visibly filter NSFAS accreditation; require administrator review.
Existing marketplaces already provide useful search and contact	DigsConnect describes verified listings, search and direct contact [4].	Do not copy blindly; focus differentiation on transparent states, visual evidence and response timing.
Landlord economics affect adoption	DigsConnect terms describe a fixed facilitation fee following confirmed tenancy [5].	Document a lower-friction prototype model; defer final commercial pricing to validation.
Personal documents create privacy risk	POPIA governs lawful and secure processing of personal information [6].	Collect the minimum, restrict access, validate uploads and define deletion expectations.

Feasibility Study

Table 8. Feasibility assessment

Dimension	Assessment	Conclusion / control
Technical	The team can build the required workflows with PHP, MySQL, HTML/CSS and JavaScript. XAMPP supports local deployment. Payment and identity integrations are beyond a safe student prototype.	Feasible when escrow and external integrations are simulated behind clear interfaces.
Operational	Students and landlords already use web and mobile channels. The workflow reduces informal hand-offs but requires administrator verification capacity.	Feasible with simple onboarding, status labels and an administrator queue.
Economic	The academic build uses open-source software and existing student devices. Production costs would be dominated by skilled labour, security, hosting and support.	Feasible for assessment; production requires staged funding and a validated revenue model.
Schedule	Five fixed due dates constrain the order of evidence. Parallel sprint work is needed between phase gates.	Feasible with the WBS, critical path, weekly integration and scope freeze after design.
Legal / ethical	POPIA, consumer law, property practice rules and payment regulation may apply. Real document and money handling would be high risk.	Prototype uses demo data and simulated funds; production requires professional compliance review.

Project Charter and Scope

Project charter

Deliver a demonstrable LAMP web application and complete SDLC evidence for Group 55 by 9 November 2026. The product shall help students discover verified accommodation, submit or instantly confirm a booking, simulate a protected deposit and confirm check-in, while landlords and administrators manage their responsibilities through role-specific dashboards.

Table 9. Scope boundary

In scope	Out of scope
Student, landlord and administrator registration/authentication	Real payment collection, custody, settlement or refunds
Verified property search, filters and rich listing details	Live NSFAS, university, identity-provider or bank integration
Standard booking with 12-hour response deadline	Legally binding digital lease execution
Instant Book for approved properties	Production-grade chat, SMS or WhatsApp delivery
Escrow lifecycle simulator and audit records	Native mobile applications
Validated demo document upload and review	Public production hosting with real personal documents
MySQL schema, seed data, installation and tests	Automated property inspections and legal accreditation decisions

Table 10. Key assumptions and constraints

Type	Statement	Planning response
Assumption	All six members can access Windows, XAMPP, Word and MS Teams.	Keep local setup documented and maintain one shared baseline.
Assumption	Supervisor feedback is available within approximately three working days.	Submit early enough to preserve correction time.
Constraint	Final academic due dates cannot move.	Treat phase dates as hard milestones and protect the critical path.
Constraint	No licensed payment or identity service is available.	Use explicit simulators and demo evidence only.
Constraint	The final report must be approximately 50-80 pages, excluding code.	Use a unified evidence-based report with numbered figures and tables.
Constraint	Every member must contribute and present.	Assign primary ownership, peer review and speaking sections.

Work Breakdown and Responsibilities

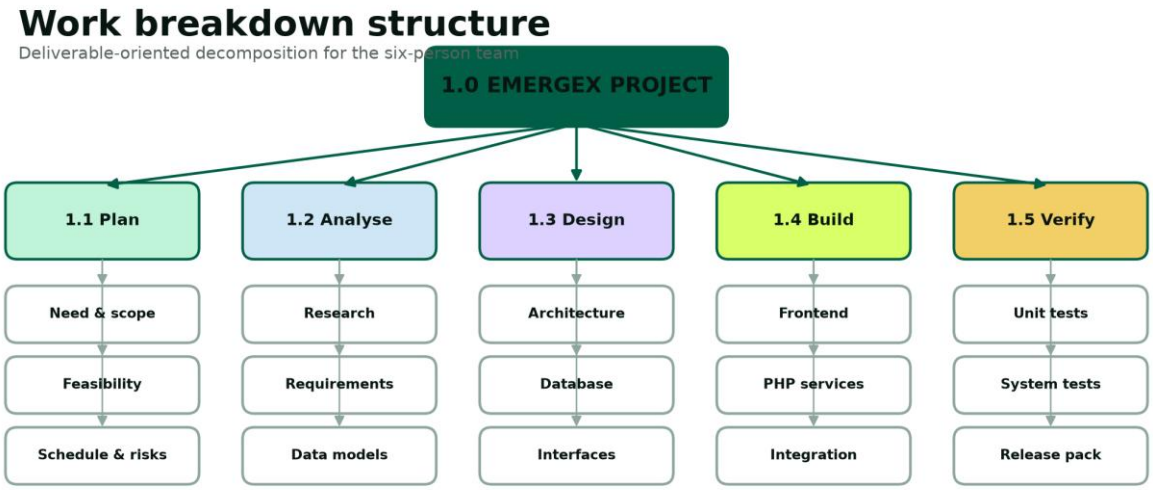


Figure 2. Deliverable-oriented work breakdown structure.

Table 11. Responsibility assignment matrix

Deliverable	Accountable	Responsible	Consulted / reviewer
Project baseline and supervisor coordination	Tristan	Tristan, Joyce	All members
Requirements and analysis	Joyce	Joyce, Lungelo	Tristan, Reenesh
UI/UX and responsive frontend	Thingo	Thingo, Tristan	Joyce, Lungelo
Database and backend services	Reenesh	Reenesh, Novithan	Tristan, Lungelo
Security and access control	Reenesh	Reenesh, Novithan	Lungelo
Testing and defect evidence	Lungelo	Lungelo, Reenesh	All feature owners
Documentation integration	Joyce	Joyce, Tristan	All members
Deployment and presentation	Tristan	Lungelo, Thingo	All members

Project Schedule

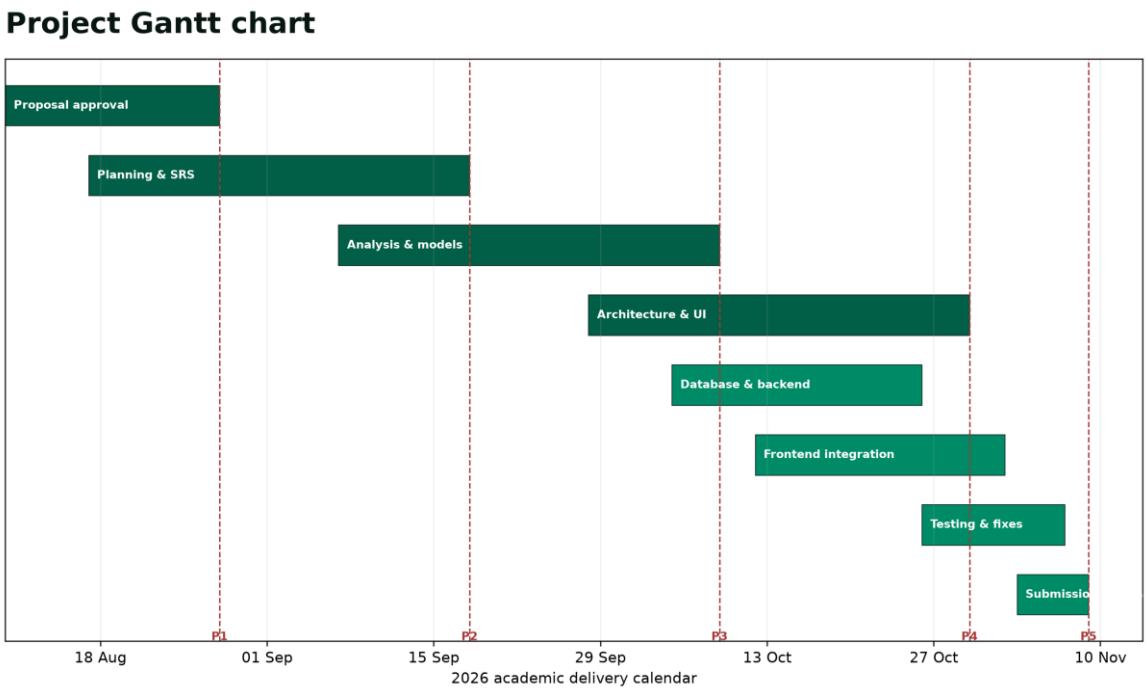


Figure 3. Gantt schedule aligned to the official semester deadlines.

Table 12. Milestone baseline

Milestone	Due date	Acceptance evidence	Owner
Phase 1 - Proposal	28 Aug 2026	Purpose, implementation language and SDLC approved	Tristan
Phase 2 - Planning	18 Sep 2026	Need, feasibility, WBS, Gantt, PERT, SRS and models	Joyce
Phase 3 - Analysis	09 Oct 2026	Methods, existing/proposed analysis and requirements	Joyce
Phase 4 - System design	30 Oct 2026	Architecture, database, pseudocode, UI and security design	Reenesh
Phase 5 - Implementation	09 Nov 2026	Code, test evidence, installation and final package	Lungelo

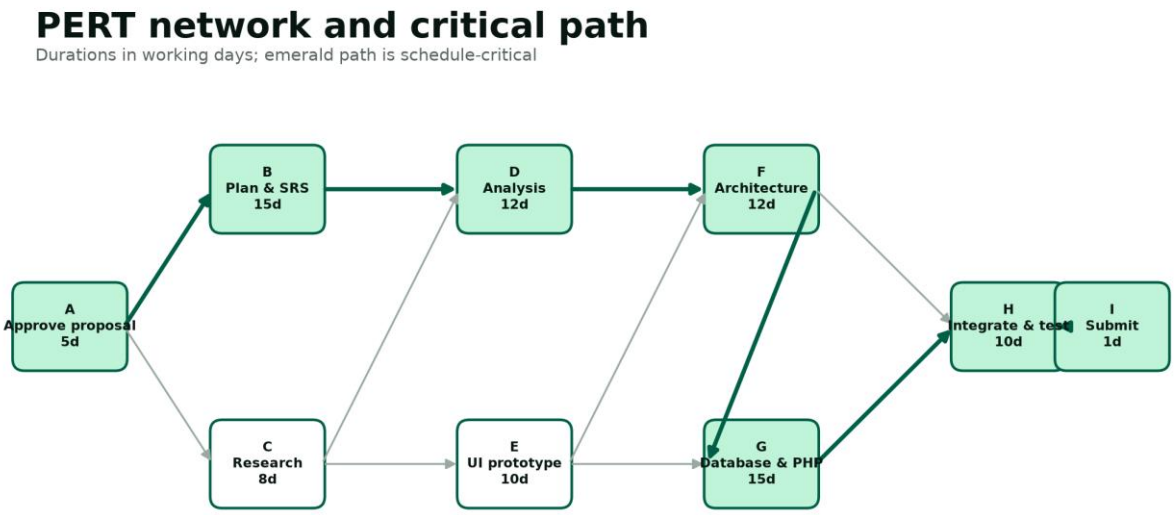


Figure 4. PERT network and critical path used for schedule control.

The critical path runs from proposal approval through planning, analysis, architecture, database/backend integration, system testing and submission. Activities such as research and early UI prototyping have limited float and may proceed in parallel, but their outputs must merge before the architecture and integration gates. If a critical activity slips, the team first reallocates members and reduces non-essential polish; it does not remove security controls, data integrity or required assessment evidence.

Risk Management

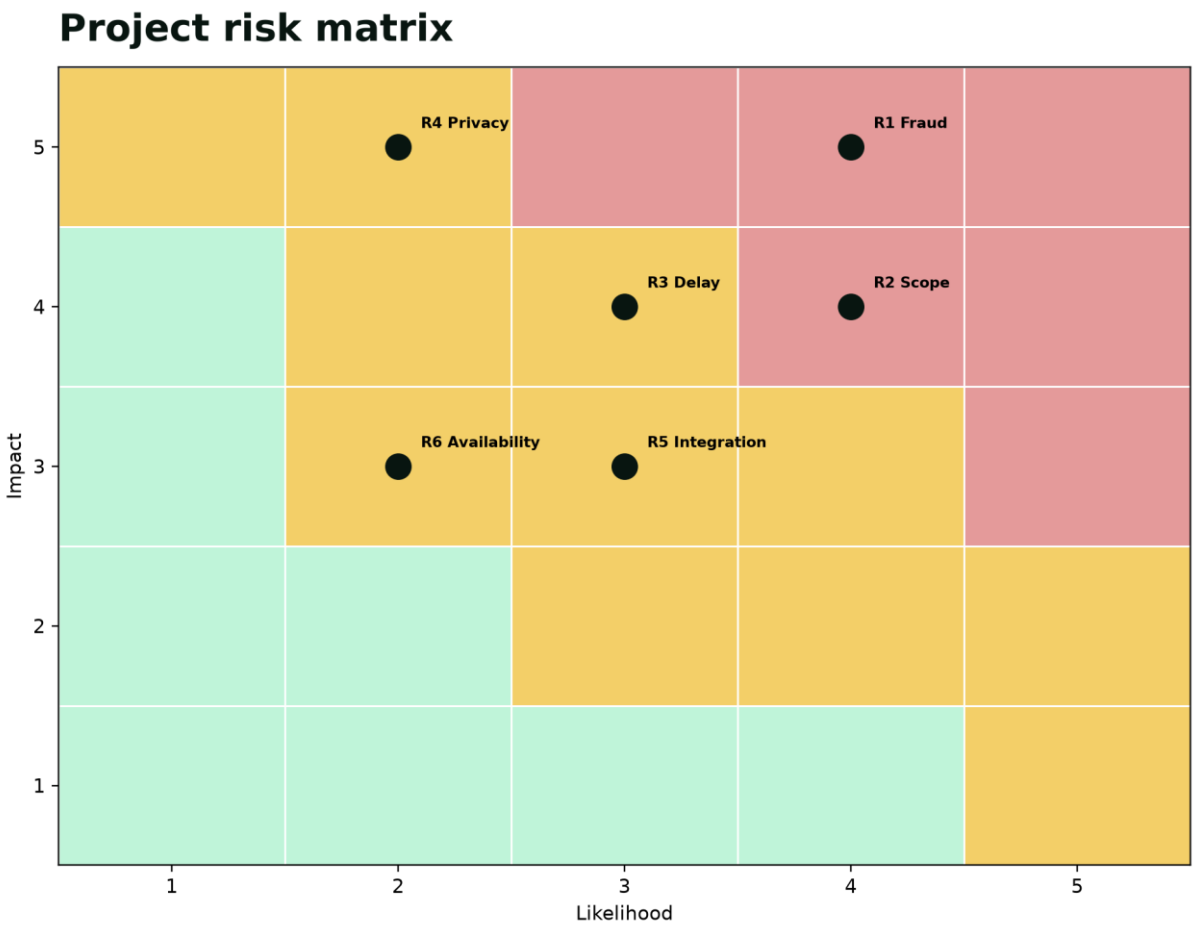


Figure 5. Likelihood-impact view of the principal project risks.

Table 13. Risk register

ID	Risk	L	I	Preventive control	Contingency / owner
R1	Fraudulent property or landlord evidence	4	5	Administrator verification and visible status; audit events	Suspend listing and preserve evidence / Admin
R2	Scope exceeds semester capacity	4	4	Freeze MVP after design; classify future features	Defer integrations and retain simulator / Project lead
R3	Critical-path delay	3	4	Weekly milestone review; parallel ownership	Reassign members and shorten polish cycle / Lead
R4	Personal-information exposure	2	5	Demo data only; minimal collection; restricted uploads	Delete affected data and investigate / Security lead
R5	Frontend/backend integration defect	3	3	Shared data contract and early vertical slice	Rollback to last verified build / Backend lead
R6	Member availability or contribution gap	2	3	Visible owners, meeting minutes and peer review	Rebalance tasks and inform supervisor / Lead
R7	Database corruption or accidental reset	2	4	Schema in source control and backup procedure	Restore latest SQL backup / Database lead
R8	Presentation failure or network interruption	2	3	Local deck, offline demo screenshots and rehearsal	Use recorded walkthrough / Presentation lead

Communication, Quality and Change Control

Table 14. Communication plan

Forum	Frequency	Participants	Purpose / record
Team stand-up	Twice weekly	All members	Progress, blockers, next actions; brief minutes
Integration review	Weekly	Developers, QA, analyst	Demonstrate vertical slice and log defects
Supervisor review	Per phase and when blocked	Project lead plus relevant owners	Approval, scope guidance and feedback record
Document peer review	Before every submission	Two reviewers not primary author	Rubric traceability, consistency and source check
Presentation rehearsal	Twice before panel	All members	Timing, handovers, individual questions and backup plan

A change request states the reason, affected requirement, scope impact, schedule impact, data/security effect, proposed decision and owner. The project lead may approve minor changes that do not alter a phase baseline. Material changes require the group and supervisor. Every accepted change updates the requirement traceability matrix and relevant test cases.

Software Requirements Specification Summary

The SRS is the controlled agreement between the approved need and the design. Requirements use stable identifiers so implementation and tests can be traced without relying on page location. The full functional and non-functional specification is developed in the analysis chapter; the planning baseline establishes user classes, interfaces and priority.

Table 15. User classes

User class	Characteristics	Primary goals	Privilege boundary
Visitor	Unauthenticated browser user	Browse published listings and understand safety process	Cannot book, upload or access dashboards
Student	Authenticated tertiary student	Search, book, track deposit state, upload demo NSFAS proof	Can access only own bookings and documents
Landlord	Authenticated accommodation provider	Manage own listings and respond to their booking requests	Cannot verify self or view unrelated properties
Administrator	Trusted operational reviewer	Verify entities, review documents and inspect audit events	Cannot impersonate users; actions are audited

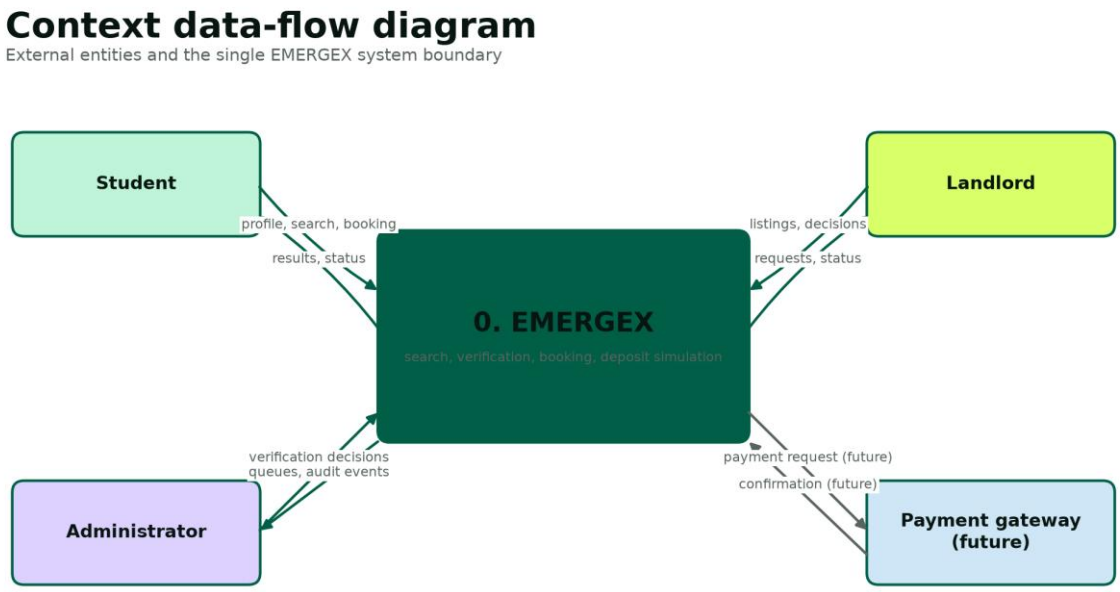


Figure 6. Context model included in the planning baseline.

Software Engineering Paradigm

The implementation follows a modular, transaction-oriented web paradigm. Presentation, application and data responsibilities are separated. Reusable bootstrap functions centralize database access, escaping, session security, CSRF, role enforcement and audit recording. Each public page performs one coherent workflow and applies server-side ownership checks. Database transactions protect multi-table booking and escrow changes. This structure is deliberately simpler than a large framework but preserves the design principles needed for maintainability and testability.

PHASE 3

Analysis Phase

Analysis validates what the system must do, why current approaches are insufficient and which data and quality rules govern the proposed solution.

Information-Gathering Methodology

The analysis uses triangulation rather than a single source. Documentary analysis covers the module guidelines, Group 55 concept notes, official DHET and NSFAS material, applicable privacy legislation and public competitor information. Structured observation maps the visible steps of searching, enquiring and confirming accommodation. Participatory workshops convert the six team members into representative student, landlord, administrator, analyst, developer and tester viewpoints. Semi-structured interview guides are prepared for future validation with real users, but no unrecorded interview claim is presented as completed evidence in this report.

Table 16. Fact-finding methods

Method	Participants / source	Questions addressed	Output
Document review	Guidelines, proposal, pitch and official sources	What is required academically, operationally and legally?	Rubric map, constraints and evidence register
Structured observation	Public accommodation search and booking journeys	Where does a user change channel, wait or lose status visibility?	Current-state process and weakness list
Participatory workshop	Six group members using assigned stakeholder roles	Which tasks, states and exceptions matter?	Use cases, business rules and priorities
Interview guide	Planned students and landlords	What information builds trust and what causes abandonment?	Validation script for supervisor-approved research
Prototype walkthrough	Team and supervisor	Can each role complete the critical path?	Usability issues, defects and acceptance decisions

Analysis of the Existing System

The relevant current environment is not one system. It includes institution residence processes, private-provider websites, accommodation marketplaces, social-media groups, direct messaging and bank transfers. A student often discovers a property in one channel, requests details in another, signs or sends documents by email and pays directly. Each channel may be individually familiar, but the overall state of the booking is difficult to audit.

DigsConnect is treated as the principal marketplace comparator because its public material describes a South African student-accommodation search and enquiry service, verified listings, direct communication and a booking/facilitation model [4][5]. The analysis does not claim that the competitor is unsafe or featureless. Instead it uses the publicly visible workflow as a baseline and asks how an academic prototype could make evidence categories, response deadlines and booking/deposit states more explicit in one interface.

Table 17. Current-state process analysis

Step	Typical channel	Observed limitation	Consequence
Discover property	Marketplace, social media or institution list	Inventory and filters vary by channel	Students repeat searches and comparisons
Verify legitimacy	Profile, messages, reviews or manual documents	Evidence standards and reviewer identity may be unclear	Trust depends on informal judgement
Inspect property	Photos, visit or separate	Media may be incomplete or inconsistent	Remote students cannot assess layout

Step	Typical channel	Observed limitation	Consequence
	video link		confidently
Enquire	Form, chat, email or WhatsApp	No common response timer or status vocabulary	Student may wait while alternatives disappear
Confirm tenancy	Message and lease exchange	Availability may not update across channels	Double-handling and stale inventory risk
Pay deposit	Direct bank transfer or external gateway	Payment may occur before safe check-in evidence	Dispute and fraud exposure
Move in	Offline inspection and handover	Booking system may not record condition or confirmation	Weak release and dispute evidence

Weaknesses and Root Causes

Table 18. Weakness analysis

Weakness	Root cause	Requirement response
Fragmented journey	Search, communication, evidence and payment are separate	Provide one booking record with visible lifecycle states.
Uneven listing quality	Free-form listings do not enforce a complete evidence set	Require structured attributes and visual-evidence categories before publication.
Slow response uncertainty	Enquiry systems may show that a message was sent but not an operational deadline	Store a 12-hour response deadline and prioritize pending requests.
Deposit exposure	Direct transfer is disconnected from verified check-in	Use a simulated hold/release lifecycle and document production PSP boundary.
Accreditation ambiguity	Status may be stated without stored review evidence	Allow administrator review and visible NSFAS filter.
Excessive personal-data handling	Documents are shared across uncontrolled channels	Minimize data, restrict uploads, validate type and define retention.
Weak accountability	Critical decisions are not consistently timestamped	Write audit events for registration, verification, booking and escrow changes.

Data Analysis: Integrity and Constraints

The central analytical decision is to treat a booking as a controlled stateful transaction, not merely a message. Rent and deposit values are copied into the booking at creation time so later property price changes do not rewrite the historical agreement. Each escrow simulator record has a one-to-one relationship with a booking. Ownership links restrict a landlord to properties they own and a student to bookings they created. Verification timestamps are separate from published status so the system can distinguish review evidence from presentation state.

Table 19. Business data rules

Rule ID	Rule	Enforcement
BR-01	Every user has one unique email address.	UNIQUE database constraint and registration error handling.
BR-02	A student number is required for student registration and is unique when present.	Server validation plus nullable UNIQUE column.
BR-03	Only active accounts may authenticate.	Login query and current-user lookup check status.
BR-04	Only verified, published properties appear in search.	Search WHERE clause requires status and verified_at.
BR-05	Rent must be positive and deposit cannot be negative.	Database CHECK constraints plus numeric input validation.
BR-06	A booking belongs to one student and one property.	Foreign keys and ownership queries.

Rule ID	Rule	Enforcement
BR-07	Each booking has exactly one escrow simulator record.	Unique booking_id in escrow_transactions; transaction at booking creation.
BR-08	Only pending standard requests may be approved or rejected.	Server-side status transition guard.
BR-09	Deposit hold is available only after approval.	Student dashboard checks booking and escrow states.
BR-10	Release occurs only after a held deposit and check-in confirmation.	Transactional state guard and audit event.
BR-11	Uploaded demo files must be PDF, JPEG or PNG and at most 5 MB.	Fileinfo MIME allow-list, size check and random stored name.
BR-12	Only administrators verify landlords, properties and documents.	Role gate and audited update actions.

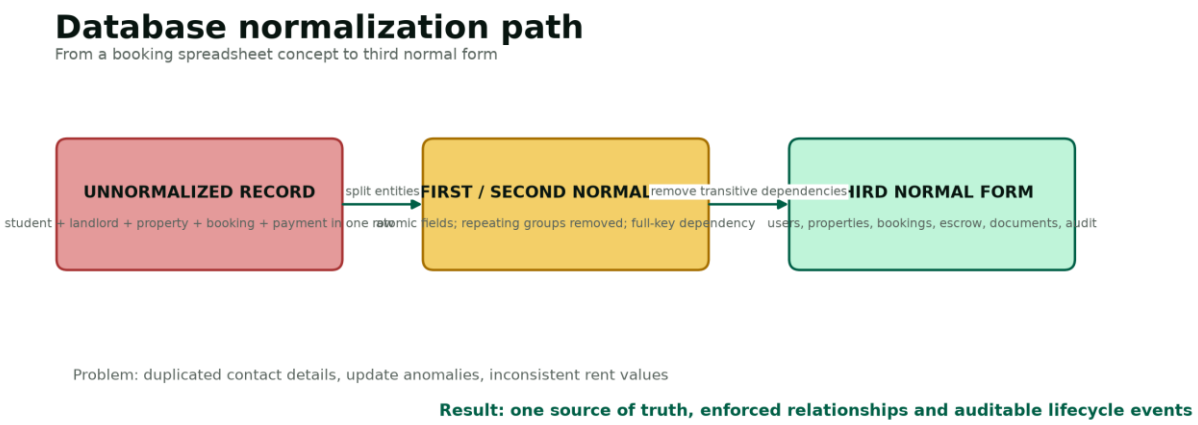


Figure 7. Normalization analysis that removes duplication and update anomalies.

Analysis of the Proposed System

The proposed system is a responsive web application with a public discovery layer and three protected role dashboards. Visitors browse published properties and safety guidance. A student account creates bookings and documents, a landlord account manages only its own property requests, and an administrator account controls verification. The application service layer validates every request, applies authorization, changes database state inside transactions where necessary and creates audit evidence. The MySQL layer enforces the relationships that remain valid even if an application page contains a defect.

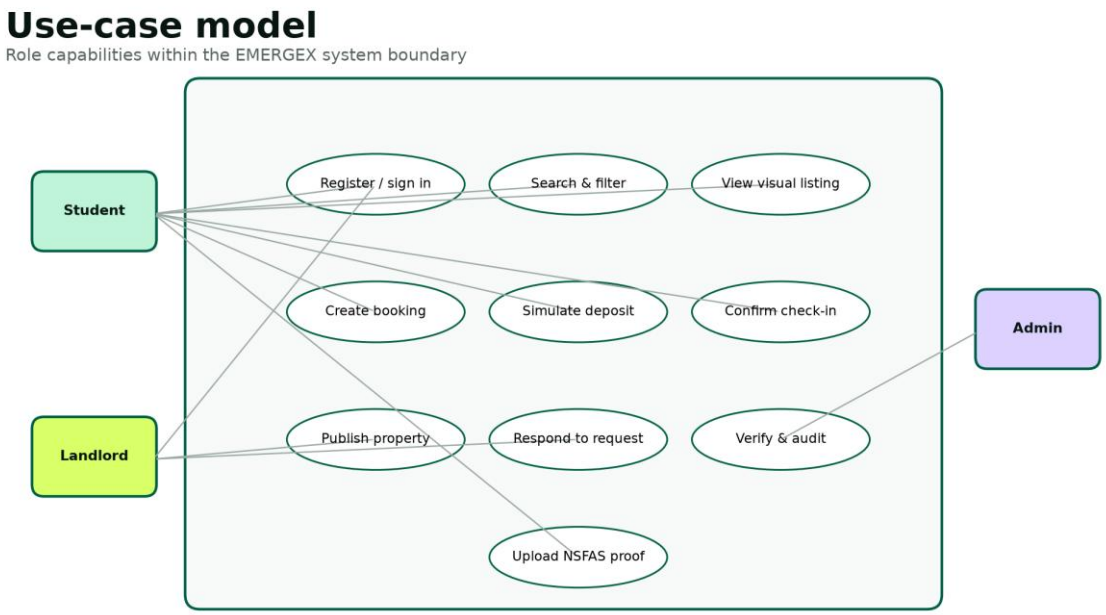


Figure 8. Role-based use-case model for the proposed system.

Functional Requirements

Table 20. Functional requirements specification

ID	Requirement	Priority
FR-01	Register a student with name, email, phone, student number and compliant password.	Must
FR-02	Register a landlord without granting verification automatically.	Must
FR-03	Authenticate an active account and rotate its session identifier.	Must
FR-04	End an authenticated session and expire the session cookie.	Must
FR-05	Display only verified, published property listings to visitors.	Must
FR-06	Search listings by title, city, suburb or nearby institution.	Must
FR-07	Filter listings by maximum price, NSFAS status and Instant Book.	Must
FR-08	Display rent, deposit, availability, address, amenities and trust badges.	Must
FR-09	Display visual evidence categories for walkthrough, floor plan and 360 tour.	Should
FR-10	Create a standard booking request for an authenticated student.	Must
FR-11	Store a 12-hour response deadline for a standard request.	Must
FR-12	Auto-approve eligible Instant Book requests.	Must
FR-13	Prevent a non-student from creating a student booking.	Must
FR-14	Allow a landlord to view requests for owned properties only.	Must
FR-15	Allow a landlord to approve or reject a pending request.	Must
FR-16	Prevent transitions from a non-pending request.	Must
FR-17	Create one escrow simulator record with every booking.	Must
FR-18	Allow a student to simulate deposit hold only after approval.	Must
FR-19	Allow release only after held status and check-in confirmation.	Must
FR-20	Display booking and deposit milestones on the student dashboard.	Must

ID	Requirement	Priority
FR-21	Upload demo NSFAS evidence using an approved file type and size.	Should
FR-22	Allow an administrator to approve or reject uploaded evidence.	Must
FR-23	Allow an administrator to verify a landlord.	Must
FR-24	Allow an administrator to verify and publish a property.	Must
FR-25	Write audit events for critical state changes.	Must
FR-26	Show administrator totals for users, properties, bookings and simulated held amount.	Should
FR-27	Provide a local command-line database installer and seed data.	Must
FR-28	Provide clear empty and database-not-installed states.	Should
FR-29	Provide responsive navigation and form layouts.	Must
FR-30	Disclose that no real funds are processed.	Must

Non-Functional Requirements

Table 21. Non-functional requirements specification

ID / category	Requirement
NFR-01 Security	All database values originating from a user shall be bound through prepared statements.
NFR-02 Security	Passwords shall be stored only as one-way password_hash outputs and verified with password_verify.
NFR-03 Security	All state-changing forms shall validate a session-bound CSRF token.
NFR-04 Security	The server shall enforce role and record ownership; hidden buttons are not authorization.
NFR-05 Privacy	Only data necessary for account, verification and booking shall be collected in the prototype.
NFR-06 Privacy	The academic prototype shall use demo documents and define deletion after assessment.
NFR-07 Performance	A seeded listing search should complete within two seconds on a local XAMPP device.
NFR-08 Availability	The local demonstration shall run without internet access after installation.
NFR-09 Usability	Critical actions and statuses shall use plain language and remain visible at mobile widths.
NFR-10 Accessibility	Pages shall use semantic headings, labels, keyboard-accessible controls and sufficient contrast.
NFR-11 Reliability	Booking plus escrow creation and hold/release updates shall be atomic database transactions.
NFR-12 Maintainability	Configuration, shared security functions, assets, database scripts and tests shall be separated.
NFR-13 Portability	The application shall run on PHP 8.1+ with PDO MySQL and Fileinfo in XAMPP.
NFR-14 Auditability	Critical events shall identify action, actor, time, entity and IP address when available.
NFR-15 Recoverability	The database shall be restorable from versioned schema/seed scripts and operational backups.

Data Modelling

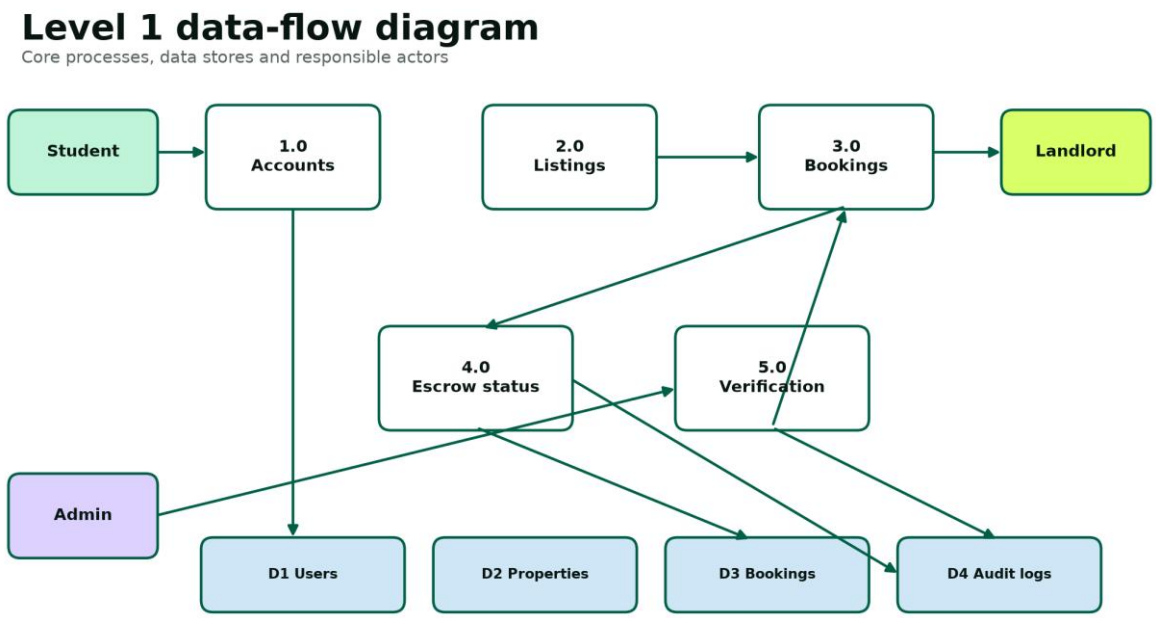


Figure 9. Level 1 data-flow model for the proposed system.

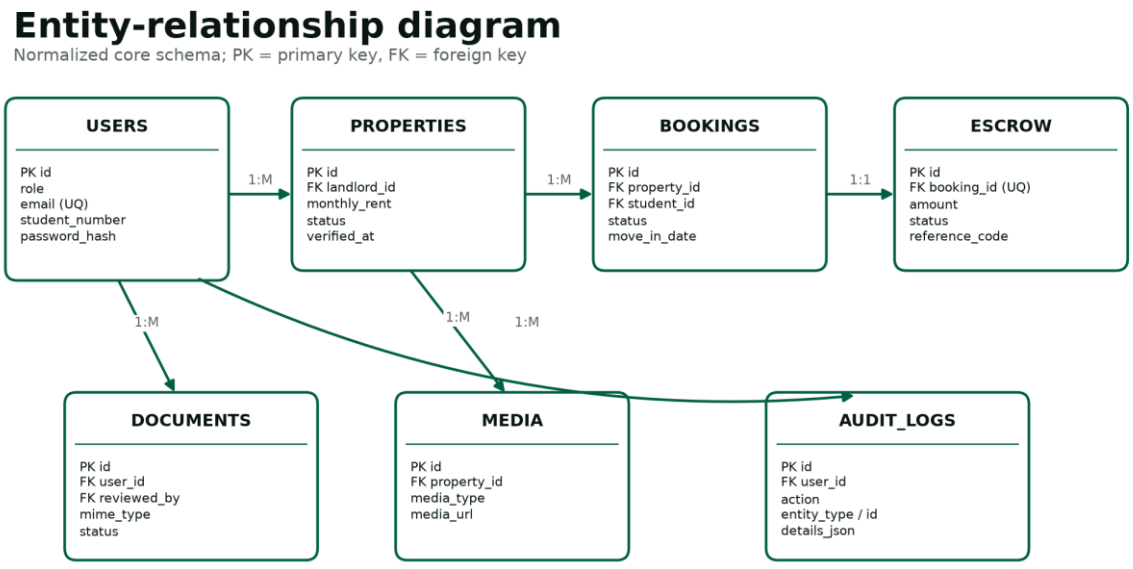


Figure 10. Entity-relationship model for the normalized database.

The data-flow model separates account, listing, booking, escrow and verification responsibilities. The ERD then allocates persistent state to seven tables. Users is the common identity source; role-specific behaviour is enforced in application logic. Properties reference the landlord owner. Bookings reference the selected property and student and capture price at the time of request. Escrow transactions record the simulated financial lifecycle. Documents preserve review metadata without exposing stored file names as public links. Audit logs remain intentionally generic so new critical actions can be recorded without changing the schema.

Requirements Validation and Traceability

Table 22. Requirements traceability sample

Requirement	Design element	Implementation location	Verification
FR-05 Verified listing search	Published + verified property rule	public/properties.php	ST-05 / ST-06
FR-11 Response deadline	Booking response_deadline	public/book.php	ST-14
FR-17 One escrow per booking	UNIQUE booking_id and DB transaction	schema.sql + book.php	DB-04 / ST-18
FR-18 Deposit hold guard	Booking state machine	public/dashboard.php	ST-19 / SEC-08
FR-21 Validated upload	Upload allow-list and protected folder	dashboard.php + .htaccess	SEC-11 to SEC-15
FR-23 Administrator verification	RBAC and audit control	public/admin.php	ST-29 / SEC-06
NFR-02 Password protection	Password API design	register.php + login.php	SEC-01 / SEC-02
NFR-11 Atomic updates	InnoDB + transactions	book.php + dashboard.php	DB-03 / ST-20

Validation uses three questions for every requirement: Is it necessary for a stakeholder or rubric item? Is it unambiguous enough to design and test? Can the delivered system demonstrate objective evidence? Requirements that fail a question are revised, deferred to future scope or marked as a production dependency rather than being described as implemented.

PHASE 4

System Design Phase

The design translates validated requirements into modular software, data, interface, network, security and test structures.

Description of the Proposed System

EMERGEX is designed as a server-rendered, three-tier web application. Public pages offer search, property comparison and safety guidance. Protected pages form three role workspaces. The design favours explicit pages and shared security helpers over a complex framework so that the academic team can trace every form action to a query, rule and test. The database remains the source of truth for identity, ownership, availability, booking state, simulated escrow and audit evidence.

Table 23. Module decomposition

Module	Responsibilities	Primary data
Bootstrap and security services	Configuration, PDO connection, session policy, headers, escaping, CSRF, authorization, audit and utilities	Configuration, users, audit_logs
Identity	Student/landlord registration, login, logout and account status	users
Discovery	Published property query, search filters, property details and visual evidence	properties, users
Booking	Standard/instant request, response deadline, landlord decision and student status	bookings, properties, users
Escrow simulator	Create transaction, hold, release and reference generation	escrow_transactions, bookings
Document review	Validated upload, administrator approval/rejection and retention status	user_documents, users
Administration	Verification queues, metrics and audit monitoring	users, properties, documents, bookings, audit_logs
Presentation assets	Responsive layout, accessible navigation, local illustrations and status treatments	CSS, JavaScript, SVG assets

Architectural Design

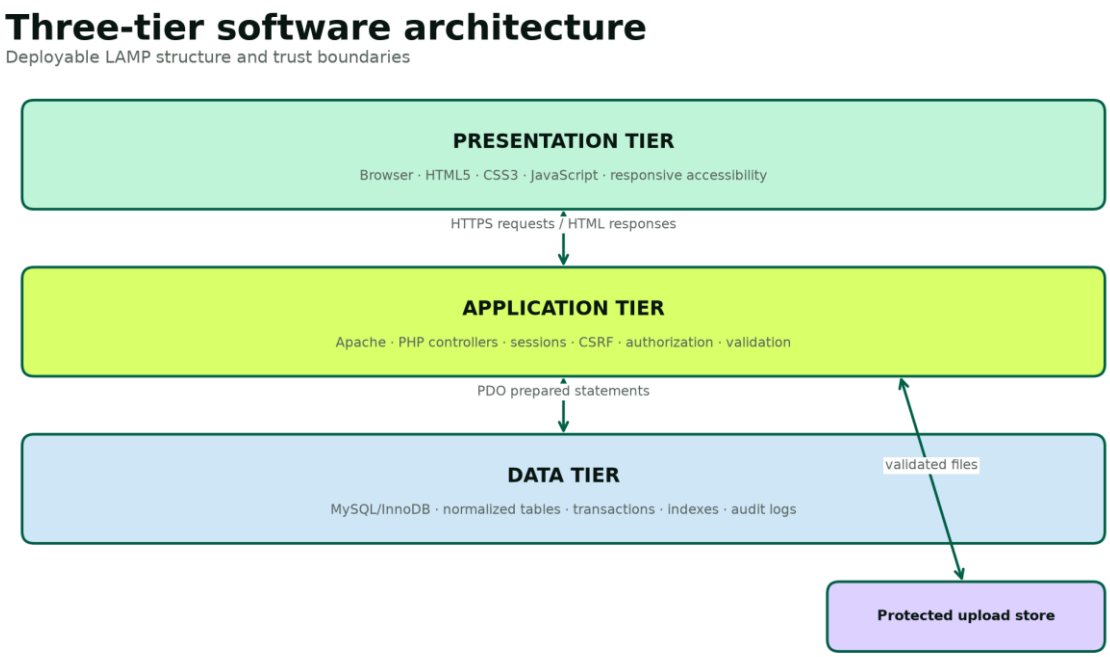


Figure 11. Three-tier LAMP software architecture.

The browser tier sends HTTPS requests and receives complete HTML responses. JavaScript is progressive enhancement for navigation and role-specific field visibility; business rules never depend on client-side code. The application tier validates requests, checks permissions and performs prepared SQL through PDO. The data tier uses InnoDB transactions and foreign keys. Uploaded demo files are stored under a protected directory with random file names, while metadata and review state remain in MySQL.

Table 24. Architecture decisions

Decision	Reason	Consequence
Server-rendered PHP pages	Minimizes build tooling and matches XAMPP teaching environment.	Simple deployment; page controllers must consistently call shared helpers.
Single bootstrap include	Centralizes security headers, sessions, database access and authorization.	One reviewed implementation controls all pages.
Native prepared statements	Separates SQL structure from user values [8].	Reduced SQL injection exposure and clearer parameter tracing.
Database transactions for multi-record changes	Booking and escrow state must not diverge.	Failure rolls back both updates.
Local SVG illustrations	Demo remains visually complete without external image calls.	Images are representative, not evidence of real properties.
No real payment integration	Avoids unsafe or misleading financial functionality.	Escrow demonstrates state and audit only.
Minimal dependencies	Improves portability and auditability for the student team.	Some framework conveniences are implemented explicitly.

Hardware and Network Architecture

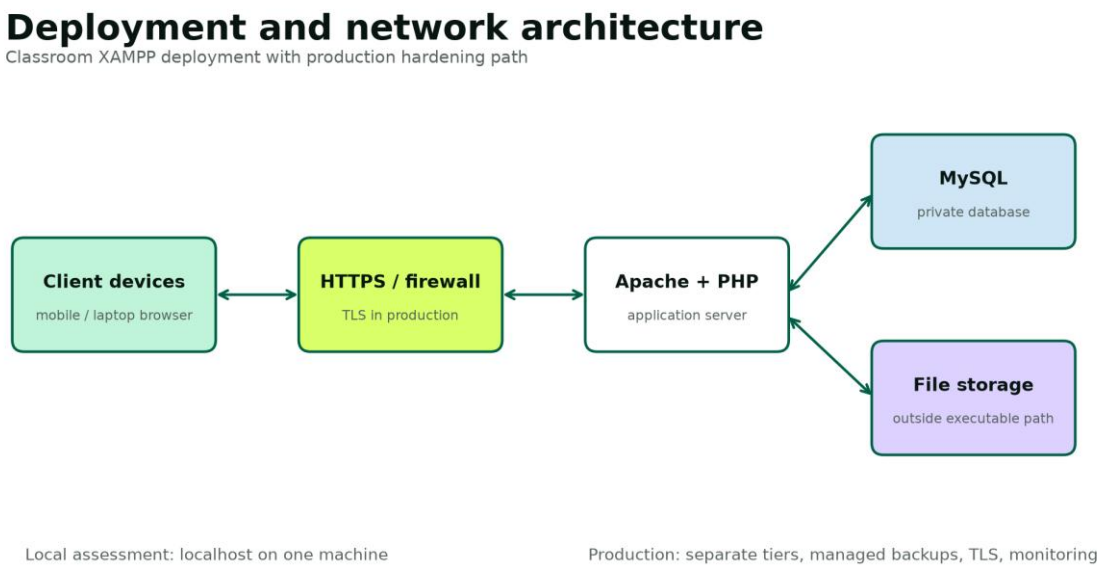


Figure 12. Deployment and network architecture.

Table 25. Deployment requirements

Environment	Minimum / proposed specification	Purpose
Developer workstation	Dual-core 64-bit processor, 8 GB RAM, 2 GB free disk, Windows 10/11	Local coding, XAMPP, Word documentation and browser testing.
Assessment host	PHP 8.1+, Apache 2.4+, MySQL/MariaDB, Fileinfo and PDO MySQL	Runs the seeded offline demonstration.
Client	Current Chrome, Edge or Firefox; 360 px minimum viewport	Student, landlord and administrator interaction.
Production application tier	Hardened Linux host or managed container, TLS, WAF, monitoring and least-privilege service account	Future secure public service.
Production database tier	Private managed MySQL, encryption, point-in-time backups and restricted network access	Future durable data and recovery.
Backup storage	Encrypted, access-controlled and geographically separate	Future recovery evidence and retention enforcement.

Class and Behaviour Design

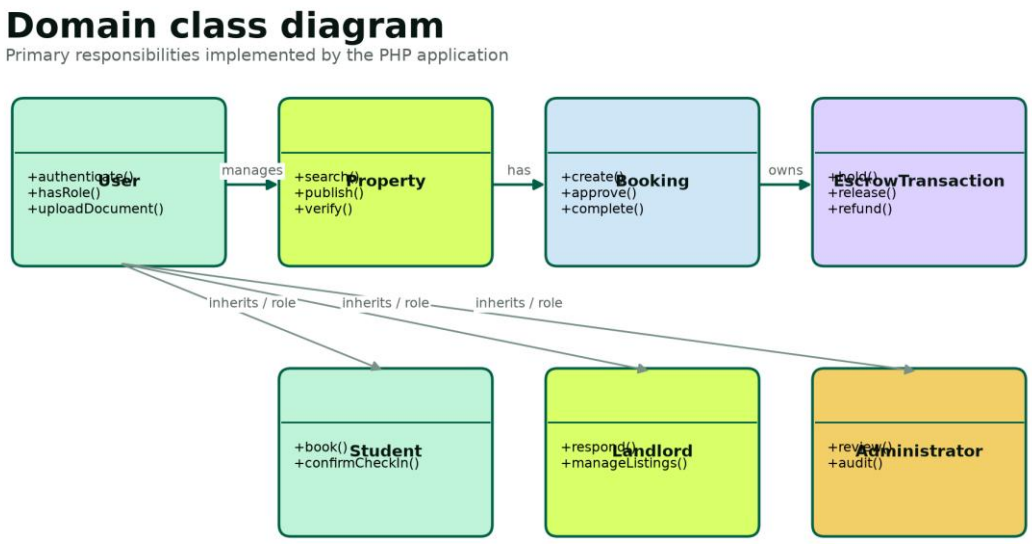


Figure 13. Domain classes and their responsibilities.

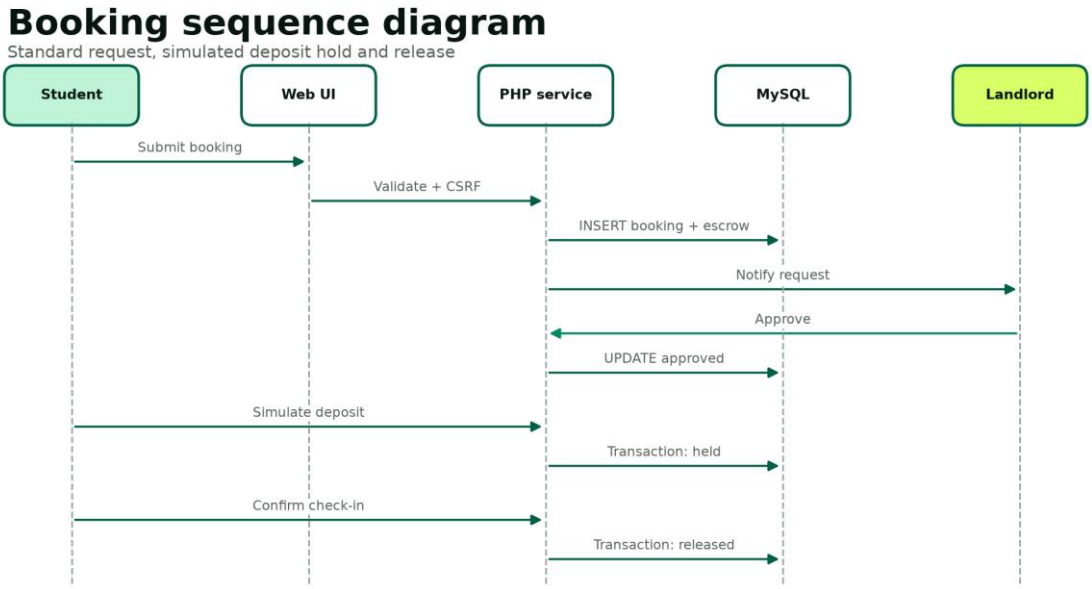


Figure 14. Booking request, approval and simulated escrow sequence.

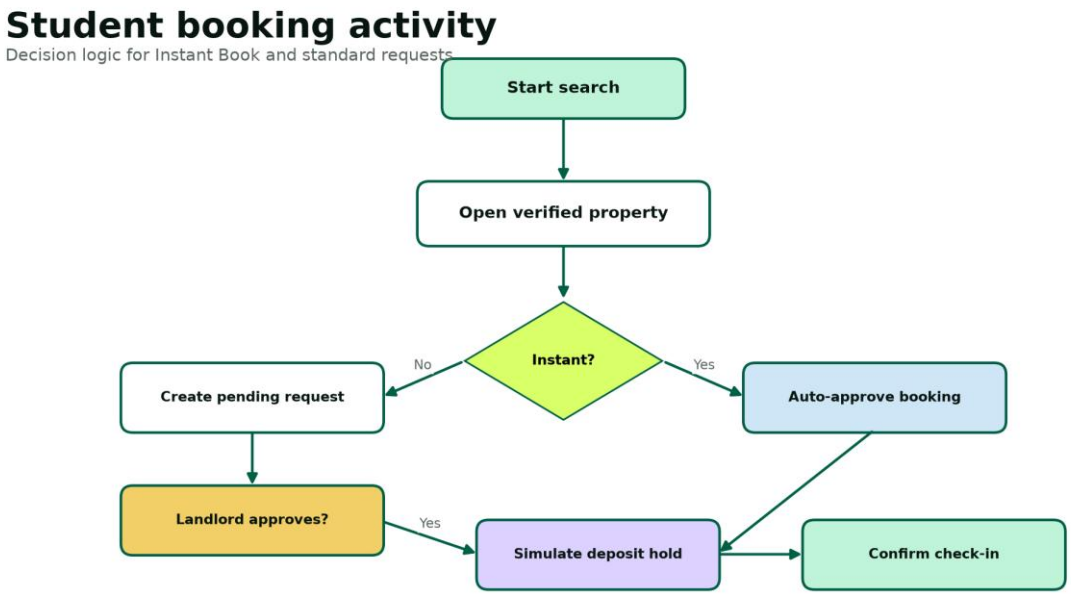


Figure 15. Student booking activity with Instant Book decision.

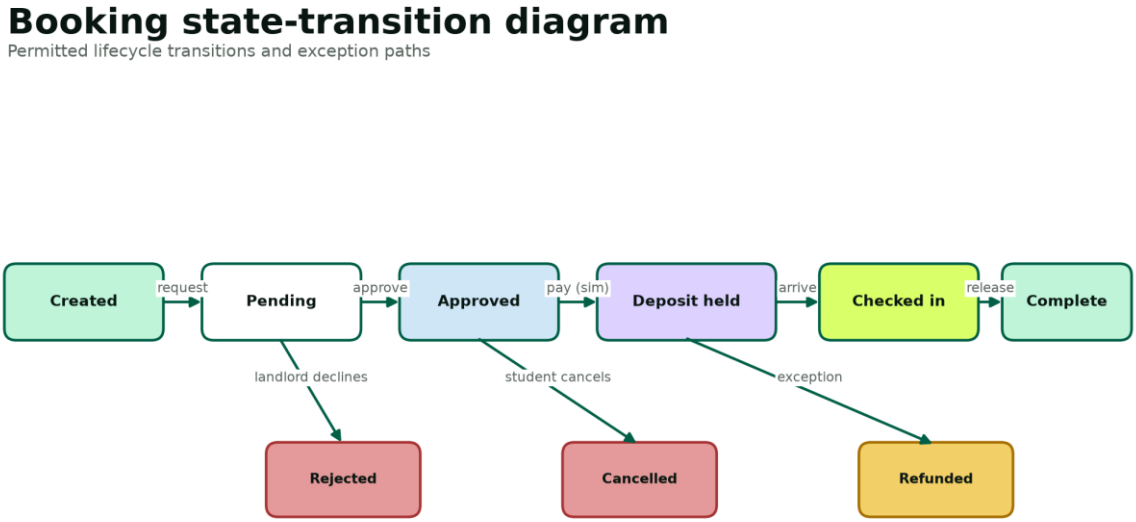


Figure 16. Permitted booking state transitions.

The state model is the design authority for booking actions. A standard request begins pending, while an Instant Book begins approved. Only pending requests can be approved or rejected. Only approved bookings can simulate a deposit hold. Only a held deposit can proceed to check-in and release. Cancellation or refund states are defined for production evolution, but the current UI implements the main successful path and landlord rejection. Every implemented transition is checked on the server and produces an audit event.

Physical and Database Design

The physical design uses seven InnoDB tables. Surrogate unsigned integer keys keep joins compact. Natural identifiers such as email, student number, escrow booking link and reference code receive unique constraints where appropriate. Foreign-key indexes support ownership and lifecycle queries. Search

indexes combine property status, city and rent, while booking indexes combine actor and status. JSON is limited to amenities and generic audit details; core relationships and states remain typed relational columns.

Table 26. Database table dictionary

Table	Purpose	Primary relationships	Retention / sensitivity
users	Authentication identity and role	Parent of properties, bookings, documents and audit	Personal; retain only for active service and legal need.
properties	Accommodation listing, price, evidence flags and publication	Belongs to landlord; parent of media and bookings	Commercial/property data; review periodically.
property_media	Typed visual references and captions	Belongs to property	Remove with property.
bookings	Historical request, move-in, captured price and state	Belongs to property and student; parent of escrow	Transactional; retain for support/legal policy.
escrow_transactions	Simulated amount and lifecycle milestones	Exactly one per booking	Financial-like demo data; no real transaction details.
user_documents	Validated upload metadata and review outcome	Belongs to user; optionally reviewed by admin	Highly sensitive; demo only and delete after assessment.
audit_logs	Actor/action/entity/time evidence	Optional user reference	Security evidence; append-oriented and restricted.

Table 27. Selected field-level data dictionary

Entity.field	Type / size	Null?	Rule / meaning
users.id	BIGINT UNSIGNED	No	Primary key.
users.role	ENUM	No	student, landlord or admin.
users.email	VARCHAR(190)	No	Unique normalized login name.
users.student_number	VARCHAR(30)	Yes	Required for student role; unique when present.
users.password_hash	VARCHAR(255)	No	Self-contained password_hash output [7].
users.verified_at	DATETIME	Yes	Identity verification evidence time.
properties.landlord_id	BIGINT UNSIGNED	No	FK to landlord owner.
properties.monthly_rent	DECIMAL(10,2)	No	Positive current advertised rent.
properties.deposit_amount	DECIMAL(10,2)	No	Non-negative current deposit.
properties.instant_book	TINYINT(1)	No	Auto-approval eligibility.
properties.nsfas_accredited	TINYINT(1)	No	Reviewed accreditation flag.
properties.amenities_json	JSON	No	Display amenities array.
bookings.student_id	BIGINT UNSIGNED	No	FK to requesting student.
bookings.move_in_date	DATE	No	Requested date, today or later at entry.
bookings.monthly_rent	DECIMAL(10,2)	No	Historical rent captured at creation.
bookings.status	ENUM	No	Controlled lifecycle state.
bookings.response_deadline	DATETIME	Yes	Standard request creation + 12 hours.
escrow_transactions.booking_id	BIGINT UNSIGNED	No	Unique FK enforcing one-to-one.
escrow_transactions.status	ENUM	No	not_paid, held, released or refunded.
escrow_transactions.reference_code	VARCHAR(40)	Yes	Random simulator reference after hold.
user_documents.stored_name	VARCHAR(255)	No	Unique random name, not user supplied.
user_documents.mime_type	VARCHAR(100)	No	Server-inspected type.
audit_logs.action	VARCHAR(100)	No	Stable machine-readable event name.

Entity.field	Type / size	Null?	Rule / meaning
audit_logs.details_json	JSON	Yes	Non-secret contextual details.

Normalization

First normal form is achieved by atomic user, property and booking columns; repeating property media records are stored separately. Second normal form is satisfied because non-key attributes depend on their complete table key. Third normal form removes landlord details from properties and student details from bookings: those facts depend on user identity and are joined through foreign keys. Escrow status is not stored as a property or user attribute because it depends on one booking. Historical rent and deposit are intentionally copied into a booking; this is a transaction snapshot, not accidental denormalization.

Table 28. Integrity and constraint design

Control	Database implementation	Application implementation
Entity integrity	Auto-increment primary key on every table.	IDs are parsed as integers and records must exist.
Referential integrity	Foreign keys between owners, properties, bookings, escrow, documents and audit users.	Ownership joins are included in protected actions.
Domain integrity	ENUM states, decimal types and CHECK constraints.	Allow-lists and state guards reject invalid values.
Uniqueness	Email, student number, escrow booking and stored filename constraints.	Duplicate registration returns a safe message.
Atomicity	InnoDB transactions.	Booking+escrow and hold/release updates commit or roll back together.
Confidentiality	Database remains outside public web assets.	Protected pages, escaped output and limited upload access.

Program Design and Pseudocode

The following pseudocode describes the highest-risk program paths. The delivered PHP uses the same validation, authorization, state and transaction order.

```
CREATE BOOKING
Require authenticated student
Validate CSRF token and property_id
Load verified, published property
Validate move-in date and message length
Begin database transaction
  status := approved if Instant Book else pending
  deadline := null if Instant Book else now + 12 hours
  insert booking with captured rent and deposit
  insert one escrow transaction with status not_paid
Commit transaction
Write audit event and redirect to student dashboard

SIMULATE DEPOSIT HOLD
Require authenticated student
Load booking + escrow WHERE booking.student_id = current_user.id
If booking.status != approved OR escrow.status != not_paid: reject
Begin transaction
  update escrow to held, paid_at=now, random reference
  update booking to deposit_held
Commit
Write audit event and show explicit no-real-funds message

CONFIRM CHECK-IN AND RELEASE
Require authenticated student and verify CSRF
Load owned booking and escrow
If booking.status != deposit_held OR escrow.status != held: reject
Begin transaction
  update escrow to released and released_at=now
  update booking to completed and checked_in_at=now
```

```
Commit
Write audit event and display completed reference

VALIDATE DOCUMENT UPLOAD
Require authenticated student and verify CSRF
Reject upload error or size > 5 MB
Inspect temporary file MIME using Fileinfo
Allow only PDF, JPEG or PNG
Generate random 40-character stored name and approved extension
Move file into protected uploads directory
Insert metadata with pending review state
Write audit event; never execute or expose original filename as a path
```

Interface Design



Figure 17. Interface design family implemented across the prototype.

Table 29. Screen design specification

Screen	Primary user task	Key controls / output	Validation and feedback
Home	Understand value and begin search	Hero search, trust pillars, featured listings and landlord call to action	Database setup state remains graceful.
Search	Narrow accommodation choices	Location, price, NSFAS and Instant Book filters	Empty state suggests widening filters.
Property	Make an informed decision	Visual proof, badges, rent, deposit, availability, amenities and booking action	Only published properties resolve.
Registration / login	Create or access role account	Labeled fields, role choice and password help	Server validation; generic login errors and throttling.
Student dashboard	Track and advance owned bookings	Status label, move-in facts, deposit/check-in actions and document upload	Actions appear only for valid states; server rechecks.
Landlord dashboard	Respond to owned property requests	Pending queue, message, approve/decline and metrics	Ownership join and pending-state guard.
Administrator dashboard	Review trust evidence	Verification queues, document decisions, totals and audit feed	Admin role gate and audited decisions.

Input Validation Design

Table 30. Input validation matrix

Input	Client constraint	Server constraint	Failure response
Email	email input type	FILTER_VALIDATE_EMAIL; normalized lowercase; unique	Specific registration duplicate, generic login error
Password	Help text and required field	At least 10 chars with upper, lower and number	Return policy message without echoing value
Phone	Required text	9-20 approved phone characters	Return field-level message
Student number	Shown/required for student	Non-empty for student; unique when present	Reject registration
Property ID	Hidden/query integer	FILTER_VALIDATE_INT; load approved record	404 or invalid action
Move-in date	date input with minimum today	Strict date parse and not before today	Return booking error
Booking message	maxlength 1000	Trim and enforce <=1000 characters	Return booking error
Filter price	number min/step	Validated float and bound parameter	Ignore invalid optional filter
NSFAS upload	accept attribute	Upload error, <=5 MB, Fileinfo MIME allow-list	Safe error and no metadata insert
Action/state	Hidden action token	Allow-list, role, ownership and current state	400/403 and no state change

Output and Report Design

All dynamic output is escaped before insertion into HTML. Dashboards report human-readable state labels while preserving machine-readable states in the database. The administrator dashboard provides operational totals and recent audit activity. The final report includes sample booking, property and verification outputs; production evolution can export filtered CSV or PDF reports without changing the transactional core.

Security and Backup Design

Table 31. Security control design

Threat / concern	Preventive control	Detective / recovery control
SQL injection	PDO prepared statements and allow-listed query structure [8].	Database error logging without showing SQL to users.
Password disclosure	password_hash/password_verify; never store plaintext [7].	Force reset and investigate account events in production.
Session theft/fixation	HttpOnly, SameSite, Secure-on-HTTPS cookies; ID rotation; OWASP-aligned session practice [9].	Logout destruction, timeout policy and event monitoring.
CSRF	Random session token on state-changing forms; hash_equals verification.	Reject with 419 response and log repeated production failures.
Broken access control	Server role and ownership queries on every protected action.	Audit admin and escrow state changes.
XSS	Contextual htmlspecialchars output and restrictive Content Security Policy.	Security review and browser console monitoring.
Malicious upload	Size/MIME allow-list, random name, blocked executable extensions.	Quarantine/reject, review metadata and delete demo files.
Personal data over-collection	Purpose limitation, minimal prototype fields and demo-only documents [6].	Retention review, deletion process and access audit.
Data loss	Versioned schema, regular encrypted backup and restore testing.	Recovery point/time objectives and incident runbook.
Fraudulent verification	Separate admin decision and visible timestamp.	Suspension, evidence preservation and audit review.

Table 32. Backup and recovery plan

Asset	Frequency	Method	Retention / restore test
MySQL production database	Daily full + frequent incremental/binlog	Encrypted automated backup to separate account/region	30 daily, 12 monthly; quarterly restore test
Uploaded evidence	Daily encrypted object replication	Versioned private object storage	Policy-driven; delete when purpose ends
Application code	Every accepted change	Version-control remote plus release archive	Retain tagged academic and production releases
Configuration secrets	On change	Managed secret store; never in repository	Versioned access policy; rotation drill
Academic local demo	Before presentation and submission	Zip package plus SQL schema/seed and offline deck	Keep assessment copy; reset demo after use

Design Test Cases

Test cases are designed before implementation completion so acceptance is based on requirements rather than on whatever the code happens to do. Each case identifies preconditions, input/action, expected result and evidence. The complete execution register appears in Phase 5.

Table 33. Representative design-stage test cases

ID	Requirement	Scenario	Expected result
TC-D01	FR-03	Sign in with correct active demo account.	Session ID rotates and correct role dashboard opens.
TC-D02	FR-05	Search while an unverified draft property exists.	Draft property never appears.
TC-D03	FR-11	Create standard request at a known time.	Pending booking stores deadline exactly 12 hours later.
TC-D04	FR-12	Book an eligible Instant Book listing.	Booking begins approved and escrow begins not_paid.
TC-D05	FR-14	Landlord changes booking ID to another owner.	No record is returned and action fails.
TC-D06	FR-18	Try deposit hold before approval.	State remains unchanged.
TC-D07	FR-19	Confirm check-in after valid hold.	Booking completes and escrow releases in one transaction.
TC-D08	FR-21	Upload a renamed executable file.	MIME/type validation rejects the file.
TC-D09	FR-23	Student posts administrator verification action.	Role gate returns forbidden; no record changes.
TC-D10	NFR-03	Submit form with missing/incorrect CSRF token.	419 response and no state change.

PHASE 5

Implementation Phase

Implementation realizes the approved design as a complete local LAMP prototype, database package, seeded demonstration and verification assets.

Implementation Overview

The delivered application is located in the application folder and is organized into configuration, shared application services, public web pages, database scripts, protected uploads, assets and tests. The implementation follows a vertical-slice approach: authentication, listing search, booking creation, landlord response, escrow simulation, document review, audit monitoring and operational reporting each include interface, validation, authorization and persistence logic. The README provides installation, demonstration accounts, backup and restore commands, and a scripted assessment journey.

Table 34. Implemented feature inventory

Area	Delivered capability	Evidence location
Public discovery	Landing page, safety explanation, search/filter, property details and responsive navigation	public/index.php, properties.php, property.php, how-it-works.php
Identity	Student/landlord registration, role routing, login throttle, session rotation and logout	public/register.php, login.php, logout.php
Student workflow	Booking creation, Instant Book, 12-hour request, deposit hold, check-in and upload	book.php, dashboard.php
Landlord workflow	Owned property summary and pending booking approval/rejection	landlord.php
Administration	Landlord/property verification, document decisions, metrics and recent audit	admin.php
Operational reporting	Administrator-only booking, property and audit reports; print/PDF control and CSV exports	reports.php and assets/css/reports.css
Security services	Prepared PDO, CSRF, escaping, headers, RBAC, upload validation and audit helper	app/includes/bootstrap.php
Database	Seven normalized tables, constraints, indexes, seed and transactional installer	database/schema.sql, seed.sql, install.php
Recovery	Timestamped database backup and guarded restore scripts with demonstrated restore evidence	runtime-tools/BACKUP_EMERGEX.ps1 and RESTORE_EMERGEX.ps1
Presentation	Deep Emerald responsive style, local SVG property assets and progressive JavaScript	public/assets
Verification	Static security checks, acceptance evidence and information-session controls	tests/security_checks.php and test-evidence

Coding Standards and Efficiency

Table 35. Coding standard

Standard	Application
Strict typing	Every PHP file begins with declare(strict_types=1).
Naming	Functions and variables use descriptive snake_case or camelCase consistently; SQL states use stable lowercase identifiers.
Separation of concerns	Configuration, shared services, layout, assets, database installation and workflow pages are separated.
Input handling	External values are trimmed, type-validated and allow-listed before use.
Output handling	Dynamic HTML is escaped through the shared e() helper.

Standard	Application
Database access	Prepared statements bind values; multi-record state changes use transactions.
Authorization	Protected actions load records through role and ownership predicates.
Error handling	Expected validation creates safe user messages; detailed internals are not exposed.
Comments	Comments explain security boundaries and non-obvious implementation choices, not restate syntax.
Efficiency	Search uses indexed fields; dashboards use bounded queries; static assets are local and small.

Representative Code Segments

The complete source is delivered separately. Short extracts below illustrate the security and transaction patterns without turning the report into a code listing.

```
// Native prepared statement with bound user input
$stmt = db()->prepare('SELECT id, password_hash, status FROM users WHERE email = ? LIMIT 1');
$stmt->execute([$email]);
$account = $stmt->fetch();

// Constant-time CSRF verification
$provided = $_POST['csrf_token'] ?? '';
if (!is_string($provided) || !hash_equals(csrf_token(), $provided)) {
    http_response_code(419);
    exit('The form expired.');
}

// Transaction protects escrow and booking state together
$pdo->beginTransaction();
$pdo->prepare("UPDATE escrow_transactions SET status='held' ...")->execute([...]);
$pdo->prepare("UPDATE bookings SET status='deposit_held' ...")->execute([$bookingId]);
$pdo->commit();
```

Error Handling and Validation

Validation failures are treated as normal user-facing outcomes; they return a precise next step without revealing server details. Authorization failures return an appropriate 403, missing records return 404, invalid state transitions return 400 and CSRF failure returns 419. Database work that changes two linked records is wrapped in a transaction and rolls back on an exception. Registration catches a duplicate-key condition and reports that the email is already used. Login always uses a generic credential error to reduce account enumeration.

Table 36. Implemented exception paths

Path	Trigger	System response
Database not yet installed	Home or listing page cannot connect	Landing page remains usable; listing page displays setup-required empty state.
Duplicate registration	Unique email constraint violation	No account created; safe duplicate message.
Login abuse	Five failed attempts in ten minutes in the session	HTTP 429 and wait message.
Expired form	Missing or mismatched CSRF token	HTTP 419; no state change.
Unauthorized role	User opens a different role dashboard	HTTP 403.
Ownership mismatch	Landlord or student changes record ID	Protected query returns no record; action stops.
Invalid booking state	Action is not permitted from current state	HTTP 400 or no action; existing state remains.
Upload failure	Transport, size or MIME check fails	No file metadata is inserted; safe error displayed.
Transactional database error	One linked state update fails	Rollback preserves previous consistent state.

Installation

1. Install or open XAMPP with Apache, PHP 8.1 or later, MySQL/MariaDB, PDO MySQL and Fileinfo.
2. Copy the delivered application folder to C:\xampp\htdocs\emergex.
3. Start Apache and MySQL from the XAMPP Control Panel.
4. Review app/config/config.php; the included values use localhost, root, blank password and /emergex/public.
5. Open Command Prompt in C:\xampp\htdocs\emergex and run C:\xampp\php\php.exe database\install.php.
6. Browse to http://localhost/emergex/public/.
7. Sign in with the seeded student, landlord and administrator accounts documented in README.md.
8. Run C:\xampp\php\php.exe tests\security_checks.php and record the output in the execution register.
9. Open the administrator Reports page, print or save the operational report as PDF, and export each CSV evidence file.
10. Run runtime-tools\BACKUP_EMERGEX.ps1, verify the timestamped SQL file, and demonstrate RESTORE_EMERGEX.ps1 -SqlFile <path> -ConfirmRestore only against the assessment database.

Table 37. Seeded demonstration accounts

Role	Email	Password	First demonstration action
Student	student@emergex.test	Student@55	Open an Instant Book property and complete the simulated lifecycle.
Landlord	landlord@emergex.test	Landlord@55	Approve or reject the seeded standard request.
Administrator	admin@emergex.test	Admin@55Secure	Review verification state and audit events.

Reset **warning**
Running the installer again drops and recreates the EMERGEX tables. Back up any demo data that must be retained before resetting.

Operational Demonstration Script

Table 38. Assessment demonstration sequence

Step	Actor	Action	Expected evidence
1	Team	Start XAMPP and show the seven-table database.	Local application and MariaDB are available without internet.
2	Visitor	Search by city and apply price/NSFAS/Instant filters.	Only verified published records appear; empty states are clear.
3	Student	Sign in, book an Instant property, hold the deposit and complete check-in.	Booking and escrow states update atomically with a SIM reference.
4	Landlord	Review and approve an owned standard request.	Only owned requests are visible; status and audit update.
5	Administrator	Inspect verification queues and decide a demo document.	Verification state, reviewer and audit evidence update.
6	Administrator	Open Reports, print/save the report and export CSV evidence.	Booking, property and audit outputs are generated from current records.
7	Team	Show security helpers and the 32/32 evidence register.	Prepared statements, password, CSRF, access and upload controls are traceable.
8	Team	Retrieve the timestamped SQL backup and explain the tested restore.	Backup path, guarded command and restored seven-table result are visible.

Use the separate 15-minute live demonstration runbook. All six members must attend, explain their own contribution and understand neighbouring code, database, security and recovery controls. Keep the supporting deck, SQL backup and a second offline project copy ready.

VERIFICATION

Testing and Evaluation

Testing combines structural control checks with repeatable unit, integration, system, security, usability and acceptance scenarios.

Testing Strategy

The strategy is risk-based. Authentication, access control, booking state and upload handling receive the most negative testing because failure could expose personal data or corrupt a transaction. Lower-risk presentation behaviour is checked across common viewport sizes and browsers. Static checks verify that required control patterns are present. Database tests verify constraints and atomic updates. End-to-end cases use the seeded accounts so evidence can be reproduced after every installer reset.

Table 39. Testing levels

Level	Purpose	Examples	Evidence
Static / review	Detect missing security and structure before runtime.	Prepared statements, password APIs, CSRF, upload blocking, InnoDB and foreign keys.	security_checks.php output and code review checklist.
Unit	Verify pure or narrowly scoped rules.	Password policy, status labels, money display and date validation.	Function assertions or controlled page inputs.
Database	Verify schema and transaction rules.	Unique email, FK rejection, one escrow per booking and rollback.	SQL results and before/after row counts.
Integration	Verify page, service and database interactions.	Registration, booking+escrow, landlord decision and document review.	Database snapshots and audit events.
System	Verify complete role journeys.	Search to completion, request approval and admin verification.	Screen captures and execution log.
Security	Challenge trust boundaries and invalid input.	CSRF, ID tampering, role bypass, MIME spoof and XSS payload.	HTTP response, unchanged data and audit evidence.
Usability / accessibility	Confirm understandability and operability.	Keyboard flow, labels, contrast, mobile layout and status comprehension.	Reviewer notes and corrected defects.
Acceptance	Confirm rubric and stakeholder outcomes.	All five phases, runnable product and presentation story.	Traceability matrix and supervisor sign-off.

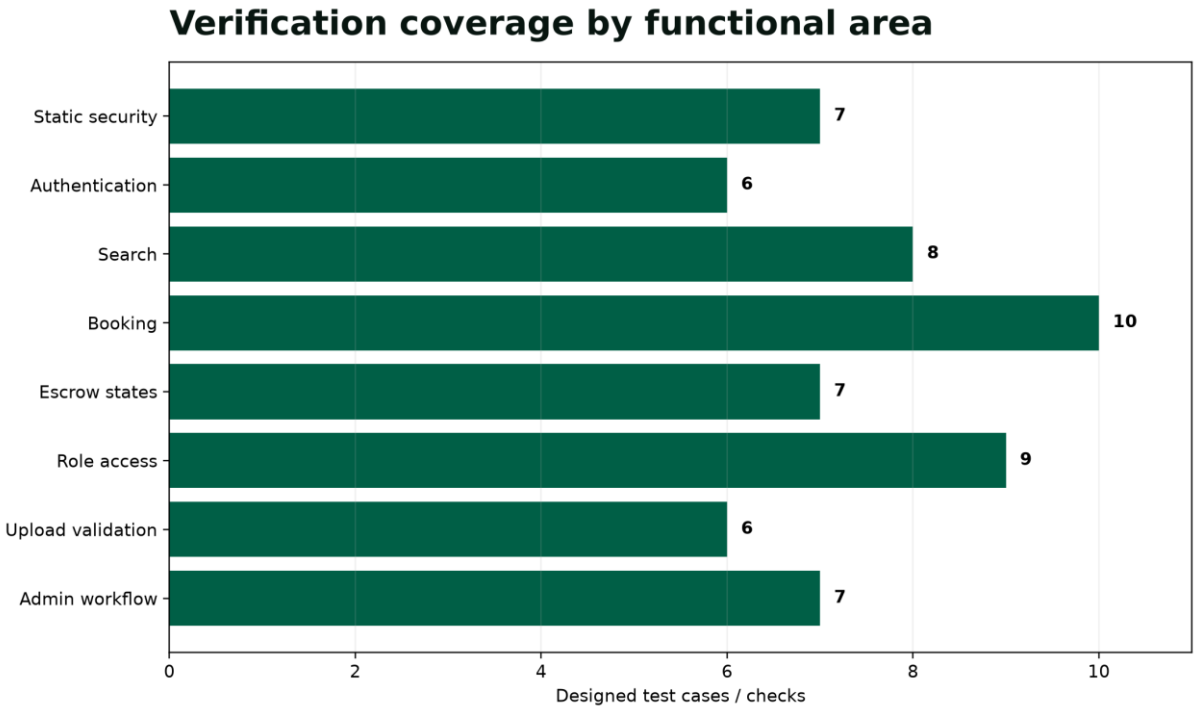


Figure 18. Designed verification coverage across functional areas.

Environment and Evidence Status

The official XAMPP 8.2.12 portable distribution supplied PHP 8.2.12 and MariaDB 10.4.32 for the execution session. All 32 documented acceptance cases passed, together with all seven structural security checks and syntax validation for all 21 PHP files. The administrator report centre returned current data, exposed print/PDF and three CSV exports, and remained responsive at 360 and 1440 px. A timestamped SQL backup was created and restored successfully to seven tables. Detailed evidence is delivered in the test-evidence folder.

The first runtime pass discovered that the NSFAS upload action was incorrectly subjected to booking lookup. The dashboard control flow was corrected so upload actions are independent while booking lookup remains mandatory for escrow actions. Upload, negative-file and full regression tests were rerun; the final verified result is 32/32 PASS. This evidence supports academic acceptance of the prototype, not a claim of production certification or regulated payment readiness.

Table 40. Runtime execution summary

Evidence	Observed result	Status
Environment	XAMPP 8.2.12; PHP 8.2.12; MariaDB 10.4.32	PASS
Install and seeded database	7 tables and 3 demonstration accounts created from a clean reset	PASS
Server/database/security cases	ST-01 to ST-29 completed with objective HTTP and SQL assertions	29/29 PASS
Browser acceptance	360, 768 and 1440 px; mobile menu; keyboard structure; local-only assets	3/3 PASS
Structural security suite	Prepared statements, password APIs, CSRF, upload block, InnoDB and foreign keys	7/7 PASS
PHP syntax validation	All application and supplied test PHP files	21/21 PASS
Operational reporting	Admin authorization, HTML output, print/PDF control and three CSV exports	PASS
Backup and restore	Timestamped SQL dump created; guarded restore returned all 7 tables	PASS

Detailed Test Register

Table 41. System and security test cases

ID	Area	Test / input	Expected result	Status
ST-01	Install	Run CLI installer on clean MySQL.	Database and demo accounts created; no SQL error.	PASS
ST-02	Register	Create valid student.	Active student created with password hash.	PASS
ST-03	Register	Reuse existing email.	No duplicate; safe message.	PASS
ST-04	Login	Use wrong password five times.	Generic errors followed by 429.	PASS
ST-05	Search	Open listing page with seed data.	Four verified properties appear.	PASS
ST-06	Search	Apply NSFAS and maximum-price filters.	Only matching verified properties appear.	PASS
ST-07	Search	Inject quote payload in q.	Request remains parameterized; no SQL error.	PASS
ST-08	Property	Request non-existent ID.	404 and no detail leak.	PASS
ST-09	AuthZ	Visitor opens student dashboard.	Redirect to login.	PASS
ST-10	AuthZ	Student opens administrator dashboard.	403 forbidden.	PASS
ST-11	CSRF	Post booking without valid token.	419 and no booking.	PASS
ST-12	Booking	Use past move-in date.	Validation error; no booking.	PASS
ST-13	Booking	Instant Book valid property.	Approved booking + not_paid escrow.	PASS
ST-14	Booking	Standard booking valid property.	Pending and deadline = creation + 12h.	PASS
ST-15	Landlord	Approve owned pending request.	Status approved; responded_at and audit written.	PASS
ST-16	Landlord	Modify request belonging to another landlord.	No record/action.	PASS
ST-17	State	Try hold while booking pending.	Rejected; state unchanged.	PASS
ST-18	State	Hold after approval.	Escrow held; booking deposit_held; SIM reference.	PASS
ST-19	State	Confirm check-in after hold.	Booking complete and escrow released.	PASS
ST-20	Atomicity	Force second update failure in test transaction.	Both updates roll back.	PASS
ST-21	Upload	Upload valid small PDF.	Random stored name and pending metadata.	PASS
ST-22	Upload	Upload >5 MB file.	Rejected; no file metadata.	PASS
ST-23	Upload	Rename PHP file to JPG.	Fileinfo rejects MIME.	PASS
ST-24	XSS	Submit script in booking message.	Dashboard displays escaped text.	PASS
ST-25	Session	Compare session ID before/after login.	Identifier changes.	PASS
ST-26	Session	Logout then revisit dashboard.	Session invalid; login required.	PASS
ST-27	Admin	Verify landlord.	verified_at set and audit written.	PASS
ST-28	Admin	Verify pending property.	Published and visible in search.	PASS
ST-29	Admin	Review document.	Status/reviewer/time update and audit.	PASS
ST-30	Responsive	Test 360, 768 and 1440 px.	No clipped controls; menu works.	PASS
ST-31	Keyboard	Complete login/search/booking without mouse.	Visible focus and logical order.	PASS
ST-32	Offline	Disable internet after local install.	Core demo and local images remain usable.	PASS

Static Security Check Results

Table 42. Completed structural checks

Check	Observed evidence	Result

Check	Observed evidence	Result
Project surface	21 PHP files and all 13 public workflow pages were found.	PASS
Relational model	7 CREATE TABLE definitions and 8 FOREIGN KEY definitions were found.	PASS
Prepared queries	37 prepare() call sites were counted across the production PHP source.	PASS
Password storage	register.php uses password_hash; login.php uses password_verify.	PASS
CSRF design	Shared token generation and hash_equals verification are present.	PASS
Upload execution block	Protected uploads .htaccess rejects PHP/PHTML/PHAR and related executable extensions.	PASS
Local assets	Four property illustrations and the logo are stored locally.	PASS
Installation evidence	Schema, seed, CLI installer, README, acceptance suite, reports, recovery scripts and demo credentials are present.	PASS

Defect Management

Table 43. Defect priority and closure rules

Severity	Definition	Response / exit rule
Critical	Data exposure, authentication bypass, corrupt transaction or application cannot start.	Stop release; fix immediately; rerun full security and critical-path regression.
High	Core role cannot complete booking, decision, verification or installation.	Fix before phase submission; rerun affected integration and system tests.
Medium	Incorrect validation, confusing state, layout break or non-critical report mismatch.	Fix within sprint; rerun affected and neighbouring cases.
Low	Minor copy, alignment or cosmetic inconsistency without task impact.	Fix during polish if risk permits; visual regression check.

Evaluation of Results

The verified execution proves the complete student-landlord-administrator journey, database constraints, negative security cases, mobile layout and installation reset for this academic build. Every final test row is backed by the JSON execution records and this report version. Future code changes require the same clean reset and regression suite; a failed case remains failed until a new execution demonstrates the correction.

OPERATIONAL ASSURANCE

System Security Measures

Security is designed as layered prevention, enforced data rules, visible verification and recoverable evidence.

Database and Data Security

The application connects through PDO with exceptions enabled and emulated prepares disabled. Dynamic values are bound as parameters in accordance with PHP guidance [8]. InnoDB foreign keys prevent orphaned business records, while transactions keep booking and escrow milestones consistent. The public web directory contains no schema or credentials; configuration is isolated under app/config. A production deployment would additionally use a dedicated least-privilege database account, TLS between tiers, encryption at rest and managed secret rotation.

User Profiles and Access Rights

Table 44. Access-control matrix

Capability	Visitor	Student	Landlord	Administrator
Browse verified properties	Yes	Yes	Yes	Yes
Register / authenticate	Yes	Yes	Yes	Seeded / controlled
Create booking	No	Owned account	No	No
View student bookings	No	Own only	Owned-property requests	Operational totals only
Change escrow simulator state	No	Own valid booking	No	Audit only
Respond to booking	No	No	Owned properties only	No
Upload NSFAS proof	No	Own only	No	No
Verify landlord/property/document	No	No	No	Yes
View audit feed	No	No	No	Yes

POPIA-Oriented Privacy Design

POPIA requires lawful, purpose-specific and appropriately secured processing of personal information [6]. The academic prototype therefore collects only account contact data, role identifiers, booking facts and optional demo evidence needed to demonstrate the workflow. It does not request bank details, real payment credentials or a comprehensive identity profile. The privacy page states the purpose and warns against real documents. Production implementation would require a complete record of processing activities, lawful-basis analysis, data-subject request process, operator contracts, incident response, retention schedule and Information Officer governance.

Table 45. Privacy lifecycle

Stage	Design control	Production completion needed
Collect	Minimal fields, plain notice and demo-only instruction.	Approved notices, consent/lawful basis and age analysis.
Use	Authentication, booking, verification and fraud-prevention purposes only.	Purpose register and automated access logging.
Store	Random upload name and restricted executable handling.	Private object storage, encryption and malware scanning.
Share	No external integration in prototype.	Operator agreements and cross-border transfer assessment.

Stage	Design control	Production completion needed
Retain	Delete demo data after assessment.	Approved schedules, legal holds and automated deletion.
Respond	Administrator can review current workflow records.	Formal access, correction, objection and breach processes.

ESTIMATION

Project Cost Estimation

The academic build uses open-source tools, while the model below estimates the production-equivalent effort required for a controlled pilot.

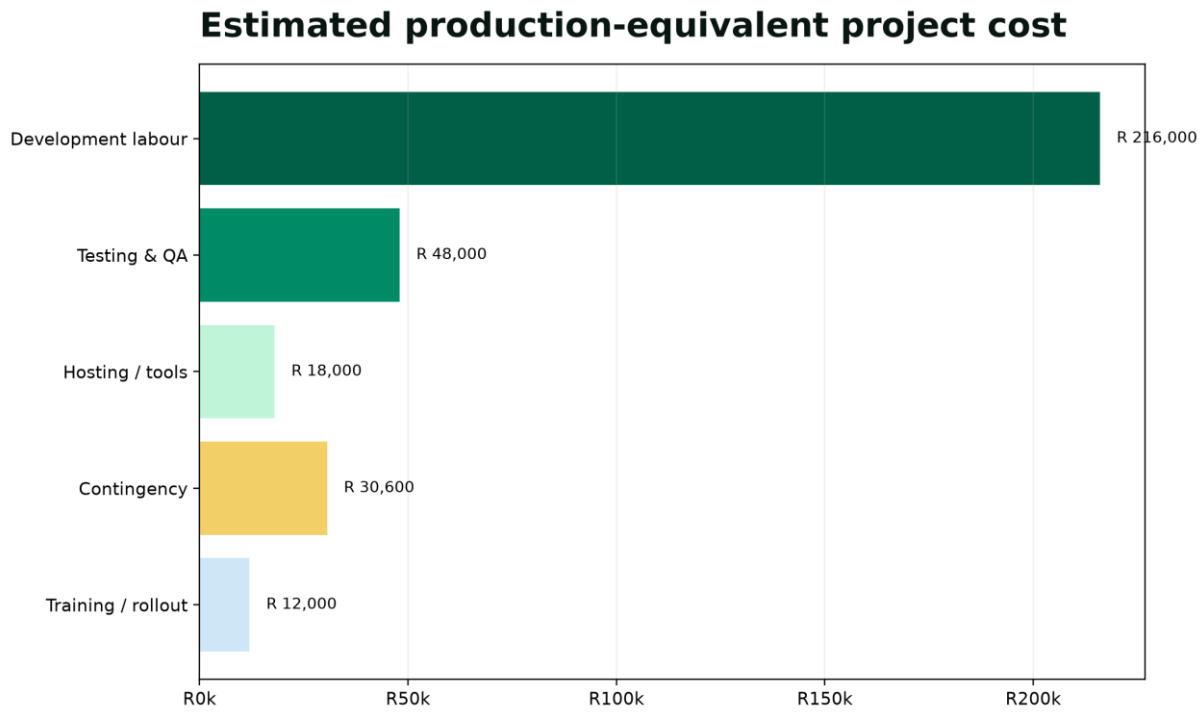


Figure 19. Production-equivalent project cost by category.

Table 46. Bottom-up cost estimate

Work package	Basis	Amount (ZAR)
Planning and analysis	240 hours × R300/hour	72,000
UX and frontend	240 hours × R300/hour	72,000
Backend and database	240 hours × R300/hour	72,000
Testing and security	160 hours × R300/hour	48,000
Hosting, domain, backups and tools	Pilot allowance	18,000
Training and rollout	Documentation and two sessions	12,000
Contingency	10% of labour and operating baseline	30,600
Total production-equivalent baseline		324,600

The estimate uses a transparent bottom-up model so assumptions can be changed. The six students do not receive the notional labour cost; it represents the opportunity cost of 880 skilled hours at an illustrative blended rate. A production project would add payment-provider fees, legal advice, penetration testing, identity/accreditation integration and ongoing support. These are excluded until pilot scope and regulatory responsibilities are validated.

Economic Benefit Model

Table 47. Illustrative benefit and break-even assumptions

Assumption	Value	Interpretation
Verified placements in pilot year	600	Illustrative, not a forecast.
Average landlord success fee	R750	Lower-friction hypothesis for validation.
Gross placement revenue	R450,000	600 × R750 before tax and operating cost.
Annual platform operating cost	R150,000	Hosting, support, verification and monitoring allowance.
Contribution before development recovery	R300,000	Revenue less operating allowance.
Indicative break-even	Approximately 13 months	R324,600 / R300,000 × 12; excludes finance and tax.

Interpretation **caution**

This is an academic feasibility model, not an investment forecast. Placement volume, pricing, VAT, payment costs, bad debt, verification labour and compliance costs must be validated before any commercial decision.

OUTPUTS

Reports and Sample Layouts

Operational reports convert transactional data into action without replacing the underlying audit trail.

Table 48. Sample booking pipeline report

Booking	Property	Student	Move-in	Rent	Deposit	Status	Response due
B-0001	Braamfontein Studio Living	Demo Student	17 Sep 2026	R 6,100	R 6,100	Pending	Within 12 hours
B-0002	Emerald House Hatfield	Demo Student	17 Sep 2026	R 5,200	R 5,200	Deposit held	Not applicable
B-0003	Durban North Student Lofts	Example Student	01 Oct 2026	R 4,550	R 3,000	Completed	Not applicable

Table 49. Sample property assurance report

Property	Landlord verified	Property verified	NSFAS	Instant Book	Rooms	Rating
Emerald House Hatfield	Yes	Yes	Yes	Yes	8	4.8
Braamfontein Studio Living	Yes	Yes	Yes	No	4	4.6
Durban North Student Lofts	Yes	Yes	No	Yes	11	4.7
Mowbray Campus Rooms	Yes	Yes	Yes	No	6	4.5

The administrator-only report centre now generates current booking pipeline, property assurance and recent audit reports. Each report can be printed or saved as PDF through the browser and exported as CSV with spreadsheet-formula injection protection. The page is server-authorized, escapes dynamic HTML, and remains responsive by keeping wide tables inside horizontally scrollable regions on small screens. Future releases can add date, city, institution, landlord and status filters plus privacy-safe aggregation.

ROADMAP

Future Scope and Enhancement

Future development should deepen assurance and integration only after the MVP is validated with students, landlords, institutions and compliance specialists.

Table 50. Prioritized roadmap

Horizon	Enhancement	Reason / dependency
Next release	Property creation/edit forms and complete media upload	Allows landlords to manage inventory without seeded SQL; requires moderation workflow.
Next release	Email/SMS notification adapter and overdue-response escalation	Makes the 12-hour service promise operational; requires provider and consent.
Pilot	Regulated payment-service integration with webhook reconciliation	Replaces simulator; requires legal, financial and security approval.
Pilot	Institution/NSFAS accreditation verification interface	Reduces manual evidence; requires official partnership/API.
Pilot	Digital lease and move-in inspection checklist	Creates stronger condition and consent evidence; requires legal template review.
Pilot	Two-factor authentication for administrators and landlords	Hardens sensitive roles.
Scale	Geospatial distance and transport-time search	Improves proximity decisions; requires maps provider and cost control.
Scale	Availability calendar, waitlist and duplicate-booking prevention	Improves multi-room inventory accuracy.
Scale	Fraud analytics and anomaly review	Prioritizes suspicious accounts without automatic unfair exclusion.
Scale	Native mobile app and offline viewing	Supports low-connectivity users after web workflow stabilizes.

Maintenance and Configuration Management

Table 51. Maintenance plan

Activity	Frequency / trigger	Owner	Evidence
Dependency and platform review	Monthly and before release	Backend lead	Supported PHP/MySQL matrix and change ticket
Security patching	Critical advisory or monthly window	Security lead	Patch record and regression result
Database backup verification	Daily backup; quarterly restore	Database lead	Restore log and recovery timing
Access review	Monthly and staff/role change	Administrator	Account/role review sign-off
Audit review	Weekly in pilot	Administrator	Reviewed anomalies and actions
Retention deletion	Monthly and purpose end	Privacy owner	Deletion log without retained sensitive content
Regression test	Every release	QA lead	Signed execution register
Documentation update	Every accepted change	Document owner	Version, change summary and traceability update

REFERENCES

Bibliography and Source Register

Sources are selected for relevance to the South African context, the comparator workflow and the implemented security controls.

[1] Department of Higher Education and Training. Report on the Ministerial Committee for the Review of the Provision of Student Housing at South African Universities (2011). https://www.dhet.gov.za/ResearchNew/39.%20Student%20housing%20at%20South%20African%20universities.pdf

[2] Department of Higher Education and Training. Student Housing Symposium keynote address (2016). https://www.dhet.gov.za/SiteAssets/Minister%20Speech%202016/Student%20Housing%20Symposium%20Minister%20Speech%2021%20July%202016.pdf

[3] National Student Financial Aid Scheme. Registering accommodation properties and student accommodation guidance. https://nsfas.org.za/content/studentacom.html

[4] DigsConnect. Student accommodation marketplace and how-it-works pages (accessed August 2026). https://www.digsconnect.com/

[5] DigsConnect. Terms and conditions, booking process and landlord facilitation fee (accessed August 2026). https://www.digsconnect.com/terms-and-conditions/

[6] Republic of South Africa. Protection of Personal Information Act 4 of 2013. https://www.justice.gov.za/legislation/acts/2013-004.pdf

[7] PHP Documentation. password_hash and password_verify guidance. https://www.php.net/manual/en/function.password-hash.php

[8] PHP Documentation. PDO::prepare and parameter binding. https://www.php.net/manual/en/pdo.prepare.php

[9] OWASP Cheat Sheet Series. Authentication and Session Management guidance. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Local project sources: IT PROJECT GUIDELINES SEMESTER 2 2026.pdf; IT Project Group List Submission.eml; IT Project 700.xlsx; group 55 files/Phase 1.docx; group 55 files/Emergex pitch.docx; and the complete 20 August 2026 IT Project 600/700 information-session recording/transcript. These files define the rubric, administration, group identity, original concept direction and final presentation controls.

APPENDIX A

Glossary

The glossary standardizes terminology used in the interface, database and assessment presentation.

Table 52. Glossary

Term	Definition
Accreditation	A formal determination by an authorized body that accommodation meets relevant requirements.
Audit log	A timestamped record of a security- or business-relevant action.
Authentication	Verification that a user controls the credentials of an account.
Authorization	Decision about what an authenticated user may access or change.
Booking	A request or reservation linking one student to one property and move-in date.
CSRF token	Unpredictable session value used to reject forged state-changing requests.
Data integrity	Accuracy, consistency and valid relationships throughout the data lifecycle.
Demo data	Artificial information safe for academic testing and presentation.
Escrow simulator	A state model that demonstrates hold and release without processing real funds.
Foreign key	Database constraint connecting a child record to an existing parent.
Instant Book	Booking path that starts approved for an eligible verified property.
MIME type	Server-inspected description of file content used in upload validation.
Normalization	Organization of relational data to reduce duplication and anomalies.
Personal information	Information relating to an identifiable person as governed by POPIA.
Prepared statement	SQL template in which values are bound separately from query structure.
Property verification	Administrator review state required before a listing is published.
RBAC	Access control based on the user role and, where relevant, record ownership.
Response deadline	Timestamp 12 hours after a standard booking request is created.
Session	Server-maintained authenticated interaction associated with a hard-to-predict identifier.
Traceability	Link from a need or requirement to design, code and test evidence.
Transaction	Atomic group of database changes that commits together or rolls back together.
UAT	User acceptance testing against agreed business outcomes.
Visual evidence	Listing media categories such as walkthrough, floor plan and 360-degree tour.
XAMPP	Local package providing Apache, MariaDB/MySQL-compatible database and PHP on Windows.

APPENDIX B

Meeting and Review Templates

Recorded coordination supports fair contribution, supervisor visibility and timely issue resolution.

Table 53. Meeting minutes template

Field	Required entry
Meeting	Team / supervisor / integration / test review
Date, time and platform	Include MS Teams link or physical/online location
Chair and minute taker	Rotate across group members
Attendance	All invited members and apologies
Previous actions	Owner, due date and completed/open state
Progress by workstream	Evidence completed since last meeting
Decisions	Decision, rationale and approver
Risks/issues	Impact, response and escalation
New actions	Action, owner and due date
Next meeting	Confirmed date and expected evidence

Table 54. Change request template

Field	Description
Change ID and requester	Unique reference and person proposing change
Reason	Problem, feedback or evidence that triggers request
Affected requirements	Stable FR/NFR identifiers
Scope and design impact	Modules, data, interfaces and documents affected
Schedule/cost impact	Critical-path and resource effect
Security/privacy impact	Threat, personal data and compliance effect
Recommendation	Approve, reject, defer or request more evidence
Decision and approver	Timestamped outcome
Verification	Tests and documents updated after approval

APPENDIX C

Final Submission Checklist

The checklist is completed by the project lead and peer reviewer before the supervisor or Moodle submission.

Table 55. Submission quality gate

Check	Owner	Evidence / sign-off
Project title, group number, supervisor and all six members appear on the first pages.	Project lead	
Phase scope matches the official deadline and rubric.	Business analyst	
Table of contents and page numbers have been updated in Word.	Document owner	
Every figure and table has a number, title and readable content.	Document reviewer	
Functional/non-functional requirements trace to design and tests.	QA lead	
Application installs from a clean XAMPP environment.	Deployment lead	
All 32 runtime tests and the report/backup controls have current evidence.	QA lead	
The administrator can generate reports, export CSV and print/save PDF.	Administrator	
A current SQL backup exists and the guarded restore has been demonstrated.	Database lead	
No real personal documents, secrets or payment data remain in the demo.	Security lead	
Source URLs and bibliography are complete and human-readable.	Business analyst	
All six members completed a timed live-demo rehearsal and can answer code/database/security questions.	Project lead	
Supervisor approval was obtained before every member submitted the same package to Moodle.	Project lead	
Final files are backed up and the submission email CCs required participants.	Project lead	